Design and Implementation of a Simulation Tool for Renewable Energy Communities: From Energy Flows to Economic Evaluation

Master's Thesis

Master's Degree in Data Science and Business Analytics

Candidate: FILIP JUREN
Supervisor: Luigi De Paoli

Academic Year 2024/25

Contents

1 Context and objectives of the simulator **1**

1.1 Context: renewable energy communities and collective self-consumption 1

1.2 Thesis objectives and scope of work 2

1.3 General assumptions and conventions 3

1.3.1 Pipeline orchestration and module responsibilities 3

1.3.2 Time convention and resolutions 5

1.3.3 Sessions, runs, reproducibility and traceability 5

2 Simulator architecture **7**

2.1 Software organisation 7

2.2 Persistence, sessions, scenarios and runs 9

2.3 Workflow description 10

2.3.1 Path resolution and session initialisation 10

2.3.2 Home and weather 10

2.3.3 Producers 10

2.3.4 Consumers 11

2.3.5 Integration bridge and materialisation of inputs 11

2.3.6 Energy balancing 12

2.3.7 Economic evaluation 12

2.3.8 Optimisation 12

2.4 Conventions and data contracts 13

2.5 References to subsequent chapters 13

3 General conditions: global configuration and climatic component **15**

3.1 Simulation session: metadata and persistence 16

3.2 Choice of the time interval and UTC convention 16

3.3 Locality selection via the Geocoding API 17

3.4 Acquisition of hourly weather data via Open-Meteo 17

3.5 Session weather outputs and feature engineering 19

3.6 Disk file organisation (session audit) 19
3.7 Visualisation and data inspection 20

4 Energy generation **21**
4.1 Purpose of the chapter and framing within the pipeline 21
4.2 Session, shared inputs and on-disk artefacts 22
4.2.1 Files relevant to generation 22
4.2.2 Module responsibilities: UI vs core 22
4.3 Workflow of the `Producers` page and configuration persistence 23
4.3.1 Producer creation/editing (CRUD) 23
4.3.2 Definition of PV areas and geometric conventions 23
4.4 Photovoltaic generation 24
4.4.1 Estimation of installable capacity: packing in the Local Area Frame 24
4.4.2 PVGIS querying and angle convention 25
4.4.3 PV generation caching 25
4.4.4 Inputs of the photovoltaic model 25
4.5 Wind generation 25
4.5.1 Technical pipeline: weather → hub-height speed → power curve 26
4.5.2 Turbine library and curve override 27
4.5.3 Wind generation caching 27
4.5.4 Inputs of the wind model 27
4.6 Produced outputs and use in subsequent modules 28
4.7 Qualitative checks and model limits 28
4.8 Figures of the chapter 29

5 Electricity demand: bottom-up modelling of consumption **31**
5.1 Model and integration in the pipeline 31
5.1.1 Framing and requirements of the demand subsystem 31
5.1.2 Modelling rationale: why this implementation 32
5.1.3 Workflow structure and produced artefacts 32
5.1.4 Generated files and output conventions 32
5.2 Demand generation modules 33
5.2.1 Base loads: always-on, standby and cycles (Baseload) 33
5.2.2 Occupancy-driven: presence, activity and lighting 34
5.2.3 Kitchen: event-driven model based on meals 35
5.2.4 Laundry: discrete cycles with scheduling constraints 36

5.2.5 Climate: compact thermo-electric model and dependence on weather . . . 37
5.3 Aggregation, scaling and interpretation of results . . . 38
5.3.1 Aggregation of the total profile . . . 38
5.3.2 Scaling to an annual energy target . . . 38
5.4 Profile examples and qualitative discussion . . . 38

6 Energy balancing: implementation, flows and shared energy 40
6.1 Role of the subsystem and data contract in the pipeline . . . 40
6.2 Scenario workspace and input persistence . . . 41
6.2.1 Filesystem structure per scenario and per member . . . 41
6.2.2 Scenario data model and legacy migration . . . 41
6.3 Time consistency and data quality . . . 42
6.3.1 Timezone normalisation and UTC convention . . . 42
6.3.2 Period definition and construction of the expected indices (T15, TH) . . . 42
6.3.3 Validations and conservative rules on missing data . . . 42
6.4 Parsing and construction of consumption and generation series . . . 42
6.4.1 15-minute consumption: format, checks and clipping . . . 42
6.4.2 Hourly generation: Totale vs Aree modes and realignment on T15 . . . 43
6.5 Member energy balance (without storage) . . . 43
6.5.1 Notation, units and kW→kWh conversion . . . 43
6.5.2 Definition of self-consumption, withdrawal and injection . . . 43
6.5.3 Hourly aggregation of flows . . . 44
6.6 Extension with electrochemical storage (BESS) in RSE-like logic . . . 44
6.6.1 Parameters, SOC constraints and efficiencies (half-cycle) . . . 44
6.6.2 15-minute simulation step and SOC update . . . 44
6.6.3 Discussion of the abstraction level (energy-based) and implications . . . 45
6.7 Community aggregation and shared energy . . . 45
6.7.1 Flow summation and definition of hourly shared energy . . . 45
6.7.2 Residuals to the grid (import/export) and consistency checks . 46
6.8 Balancing outputs and run versioning . . . 46
6.8.1 Output schema (CER, members long, summary, metadata) . . 46
6.8.2 Append-only registry, active run and configuration snapshot . 46
6.9 Targeted comparison with CACER_Simulator (RSE) . . . 47
6.10 Known limitations and extensions . . . 47

7 Economic evaluation of CER projects and BESS optimisation 48
7.1 Input data and link to the energy run 48
7.2 Economic assumptions and price/incentive profiles 49
7.2.1 Time profiles: conventions, units and validation 49
7.2.2 Hourly zonal price: definition and acquisition modes 50
7.2.3 Energy purchase price 50
7.2.4 Energy sale price 51
7.2.5 TIP incentive profile 52
7.2.6 TIAD component refund profile 53
7.2.7 Tax deduction (Bonus ristrutturazioni): parameterisation . . . 53
7.2.8 Operational remarks and limitations 54
7.3 Purchase costs, sale revenues and incentives 54
7.4 Allocation of incentives to members 55
7.5 Annual income statement and DCF model 56
7.6 Economic outputs and traceability 58
7.7 Optimisation of BESS sizing as a function of NPV 58
7.7.1 Allocation problem and assumptions 58
7.7.2 Greedy capacity-increment algorithm 59
7.7.3 Stopping criteria and outputs 59
7.7.4 Considerations 60

8 Case study: dataset construction and scenario set-up 61
8.1 Primary data collection: in-person interviews and plant survey 62
8.2 Estimation of photovoltaic generation: PVGIS and actual roof configuration 62
8.3 ARERA consumption curves as benchmark: motivation and source . 63
8.4 From the ARERA *month* × *day type* data to an annual series compatible with the simulator 64
8.4.1 Cleaning and restructuring of the files (filter + pivot) 64
8.4.2 Construction of the annual calendar and mapping of the *day type* 65
8.4.3 Join and reconstruction of the hourly series 65
8.4.4 Scaling to the meter point annual consumption: from a profile "shape" to a profile "user" 65
8.4.5 Conversion $T_H \to T_{15}$ for direct comparison 66
8.5 Expected output of the data phase and integration into the scenario . 66
8.6 Energy balancing results: comparison between ARERA profiles and simulated profiles 67

8.6.1 Community KPIs 67
8.6.2 Member KPIs and impact on prosumers 68
8.6.3 Discussion: interpretation of differences and "daytime" bias of the simulated profile 69
8.7 Economic-financial evaluation: price assumptions, incentives and assets 70
8.7.1 Purchase and sale prices 70
8.7.2 Incentives: TIP and TIAD 70
8.7.3 Assets and CAPEX/OPEX assumptions 71
8.7.4 Finance and taxation 71
8.8 Economic-financial results: comparison between simulated profiles and ARERA benchmark 72
8.8.1 Overall KPIs 72
8.8.2 Income statement (PnL): main drivers 73
8.8.3 Cash flows and decomposition of the ΔVAN 74
8.8.4 Comparison with existing simulators and chapter closing . . . 74

9 Conclusions and future developments 75
9.1 Future developments of the simulator 75
9.1.1 Generation: data acquisition and automatic 3D render generation via convolutional networks 75
9.1.2 Consumers: integration of new loads (e.g. electric vehicles) with a bottom-up approach 77
9.1.3 Optimisation: extension from the battery alone to the optimisation of PV and wind systems 77
9.2 Conclusions: usefulness of the simulator and modularity for reuse and specialised analyses 78

List of Figures

2.1 Simulator workflow: application phases, integration points and persistence on filesystem (session, scenarios and runs). 8

3.1 Hourly profile of external temperature in 2023 for Doberdò del Lago, generated by the simulator from Open-Meteo data [6]. 20

3.2 Excerpt (first 10 rows) of the session hourly dataframe with timestamp in UTC, temperature and wind variables. The objective is to show the structure of the dataset used by the simulator and the columns made available to downstream modules. 20

4.1 Workflow diagram for generating photovoltaic generation in the simulator. Orchestration is handled in `cer_app` (Streamlit UI), while geometric estimation, PVGIS querying and caching are delegated to the domain modules in `cer_core`. Caching logic avoids repeated calls when the configuration does not vary. 29

4.2 User interface for defining photovoltaic areas. The user draws an installable surface and sets, for each area, the essential geometric parameters (tilt and orientation) that drive installable-capacity estimation and the PVGIS request. 30

4.3 3D render of the roof without panels (left) and with panels positioned according to the packing logic (right). The visualisation supports qualitative validation of installable-capacity estimation, highlighting geometric constraints and spacing. 30

5.1 Conceptual scheme of the workflow within the *Consumers* page. . . . 33

5.2 Example of base-load profile in the month of January. 39

5.3 Example of occupancy-driven profile in the month of January. 39

5.4 Example of kitchen profile (event-driven) in the month of January. . . 39

5.5 Example of laundry profile in the month of January. 39

5.6 Example of climate profile in the month of January. 39

5.7 Example of aggregated profile in the month of January. 39

List of Tables

4.1 Main inputs of the photovoltaic model in the simulator. Configuration is defined in the interface and persisted in `producers.json`; the hourly series is obtained via PVGIS [7], [8]. 26

4.2 Main inputs of the wind model in the simulator. The weather source is the session dataset generated via Open-Meteo [6]; the speed-to-power conversion follows an approach based on shear and power curves [10], [11], [13]. 28

5.1 Typical outputs of the consumption subsystem in the per-consumer cache. All files are 15-minute CSVs in kW and share the same time index. 33

8.1 Main characteristics of the interviewed meter points (base year 2023). 62

8.2 Comparison of community KPIs (ARERA run vs simulated run). . . 68

8.3 Comparison of KPIs for prosumer members (ARERA run vs simulated run). 69

8.4 Comparison of overall economic-financial KPIs (ARERA run vs simulated run). 72

8.5 Income statement comparison (Year 1): ARERA run vs simulated run. 73

8.6 Sum of the main income statement items (years 1–20): ARERA run vs simulated run. 73

8.7 Decomposition of the (discounted) ΔVAN between simulated run and ARERA run. 74

Chapter 1

Context and objectives of the simulator

1.1 Context: renewable energy communities and collective self-consumption

Renewable Energy Communities (CER) and, more generally, collective self-consumption configurations fall within the framework of the energy transition and of users' active participation in the electricity system, enabling forms of coordinated generation and consumption close to the loads. At the European level, these configurations are framed by two directives that promote generation from renewable sources and the participation of citizens and energy communities in energy markets (RED II and the directive on the internal market for electricity). [1], [2]

From an operational and regulatory standpoint, a central concept is *shared energy* on an hourly basis. In Italian CERs, sharing takes place through the public distribution network and is determined, for each hour, starting from the injection and withdrawal flows of the meter points belonging to the configuration. In particular, the shared energy in the hour under consideration is defined as the minimum between (i) the energy injected into the grid overall by the generation points of the configuration and (ii) the energy withdrawn overall by the consumption meter points that are part of it. The definition is adopted in the discipline of the Testo Integrato Autoconsumo Diffuso (TIAD) and in the operating rules for access to the service, as well as in the implementing decree relating to CACER configurations. [3], [4], [5]

Before the planning and feasibility-study phase, it is useful to recall the main economic benefits associated with participation in a CER: (i) the savings linked to self-consumption, due to the reduction in energy purchased from the grid, (ii)

the remuneration of shared energy through the incentive mechanism recognised for shared kWh, and (iii) the sale of the energy injected into the grid by the CER plants according to market mechanisms (e.g. zonal price). Within this framework, the analysis of a CER requires tools capable of quantifying, with adequate time granularity, the energy flows of the individual members and of the community and of translating such flows into economic-financial results consistent with scenario assumptions (tariffs, incentives, investments). For example, the decision to introduce a storage system and its sizing depend on the interaction between generation profiles, consumption profiles and remuneration mechanisms for shared energy, making a simulator that enables comparative and sensitivity analyses useful.

1.2 Thesis objectives and scope of work

Thesis objective. The objective of this thesis is to analyse Renewable Energy Community (CER) configurations and, more generally, collective self-consumption, in order to quantify the energy and economic-financial benefits and assess their sensitivity to scenario inputs (consumption and generation profiles, tariffs and incentives, investments, member configuration, presence and size of storage).

Approach and tool. To pursue this objective, a modular simulator is developed that, starting from a scenario configuration, reconstructs energy flows and the corresponding economic flows both for each member and for the community as a whole, enabling an economic-financial assessment of it.

What the simulator enables. The simulator is structured as an end-to-end workflow (a *pipeline*)[1] in which the output of each module constitutes the input of the next. Once the reference scenario is defined, the tool allows one to:

1. generate hourly photovoltaic (FV) and wind generation profiles starting from PVGIS[2];

2. generate 15-minute consumption profiles for each member via *bottom-up* modelling[3] with controlled stochastic components[4];

[1] In the following, *pipeline* indicates a sequence of processing steps in which the output of one module constitutes the input of the next, from scenario definition up to the final results.

[2] PVGIS is the acronym for *Photovoltaic Geographical Information System*. In the context of this thesis it is used to obtain hourly estimates of renewable yield starting from weather data and location.

[3] Approach in which the aggregated profile is built starting from the contribution of individual meter points or elementary loads, instead of starting directly from an already aggregated curve.

[4] A random variability is introduced to represent behavioural heterogeneity; by fixing the *seed*

3. reconstruct the energy flows of member and community (withdrawals, injections, self-consumption, shared energy) and the related indicators;

4. evaluate economic flows and the main financial indicators (KPI)[5], including a storage sizing procedure as a function of an economic criterion.

Furthermore, the simulator is designed to support:

- **reproducibility**: possibility of replicating a simulation with the same inputs, assumptions and stochastic parameters (e.g. seed), making scenarios and sensitivity analyses comparable;

- **traceability**: structured logging of executions (*run*) and persistent saving of *session artefacts*[6] (configuration, assumptions, outputs) to enable verification and control of results;

- **modularity**: ability to implement the simulator as a set of independent modules, each dedicated to a well-defined task; an orchestration component coordinates module execution and makes it possible to extend or replace individual blocks of the pipeline without modifying the entire process.

Scope of work. The scope is deliberately focused on energy and economic models with controlled complexity, suitable for comparative scenario evaluations. Not included are: an electrical network model (voltages, currents, congestions), a detailed electrotechnical sizing (inverters, strings, clipping) and an optimal day-ahead control of storage; such extensions are discussed as future developments.

1.3 General assumptions and conventions

1.3.1 Pipeline orchestration and module responsibilities

The simulator is organised in two distinct levels:

1. **orchestration level** (`cer_app`, Streamlit interface), which guides the user along the pipeline, validates inputs and manages persistence on disk;

(initial value of the random number generator) the generation remains reproducible.

[5]KPI: *Key Performance Indicators*, synthetic indicators for comparing scenarios. In this work, among others, NPV (VAN: net present value), IRR (TIR: internal rate of return) and payback (payback period) are used.

[6]*Session artefacts* are the files and datasets produced during a specific execution of the simulator (scenario configuration, parameters/assumptions and intermediate/final outputs). Their *persistence* consists in storing them on disk so that results can be verified or reproduced even at a later time.

2. **domain level** (`cer_core`), which implements the calculation logic (weather, generation, demand, balancing, economics and optimisation).

In the following, *orchestrator* refers to a Streamlit page that: (i) collects and validates user inputs, (ii) invokes functions and models in `cer_core`, (iii) saves the produced artefacts to the filesystem, maintaining consistency and traceability of results within a *session*. An overview of the architecture is shown in Figure 2.1 (Chapter 2).

General conditions / Home orchestrator (`cer_app/app.py`). Defines *session metadata* (time interval, location, seed) and materialises the reference climate datasets. Invokes `cer_core/condizioni_generali` for geocoding and weather download/caching and saves to the session the weather files (hourly and daily series) that become canonical inputs for all subsequent phases.

Producers orchestrator (`cer_app/pages/1_Producers.py`). Manages plant configuration and generates/ingests renewable generation profiles on an hourly grid. Invokes `cer_core/produttori` (FV and wind) and, when necessary, uses the weather variables persisted by Home. The modelling details are discussed in Chapter 4.

Consumers orchestrator (`cer_app/pages/2_Consumatori.py`). Parameterises and generates 15-minute demand profiles via bottom-up modelling (base loads, appliances, occupancy and climate dependencies). Invokes `cer_core/consumatori` and uses, if required by the models, the session weather variables. The demand model is described in Chapter 5.

Balancing orchestrator (`cer_app/pages/3_Bilanciamento_CER.py`). Recomposes generation and consumption to calculate the energy flows per member and at community level (self-consumption, withdrawals, injections) and determines shared energy on an hourly basis. Invokes `cer_core/bilanciamento`, which constitutes the simulator's *energy source of truth*: this is where time alignment of the series, consistency checks and the production of energy outputs persisted as *run* take place. Balancing and the definition of shared energy are addressed in Chapter 6.

Economic evaluation orchestrator (`cer_app/pages/4_Valutazione_Economica.py`). Translates energy results into costs/revenues and financial KPI, saving a snapshot of the assumptions (tariffs, incentives, costs, discount rate) used in the evaluation. Invokes `cer_core/economics`, which reads the balancing outputs and produces a persisted and comparable *economic run*. When requested, it activates

`cer_core/optimization` to explore BESS sizing/allocation heuristics as a function of the KPI (Chapter 7).

1.3.2 Time convention and resolutions

The simulator adopts **UTC as the temporal reference for all persisted artefacts** (scenario inputs, weather cache and run outputs). This convention reduces ambiguities related to daylight saving time (DST) changes and makes it possible to align heterogeneous sources (weather, generation, consumption) consistently. In the calculation core, time indices are treated as `DatetimeIndex` *timezone-aware* and normalised to UTC.

The pipeline uses two main time grids:

- **15 minutes** for consumption profiles and for local balance calculations (e.g. intra-hour logics, including any storage dynamics);
- **hourly** for renewable generation profiles and for the community quantities defined by regulation, in particular shared energy.

When series are present on different grids, alignment takes place in the balancing module according to a deterministic rule. In particular, hourly production is brought to the 15-minute grid through a piecewise-constant expansion (replicating the hourly value over the four sub-intervals), so as to enable the calculation of local flows on the T15 grid without introducing artificial energy. The quantities related to sharing, instead, are calculated and reported on an *hourly* basis.

1.3.3 Sessions, runs, reproducibility and traceability

Each simulation is organised in a *session*, i.e. a folder on disk that gathers the data needed to: (i) reconstruct input choices (scenario), (ii) reuse intermediate datasets (e.g. weather and generated series), (iii) robustly compare results produced in subsequent executions.

A session typically contains:

- **metadata and configuration**: time interval, location and seed (as well as producer/consumer configurations and scenario parameters);
- **persisted climate datasets** (hourly and daily series) used as common input of the pipeline;
- **section outputs**: files produced by the orchestrators (hourly production per member/plant, 15-minute consumption per member);

- **energy runs** and **economic runs**: folders/artefacts generated respectively by balancing and economics, each accompanied by output files and execution metadata.

Reproducibility. Reproducibility concerns the possibility of regenerating the same results given the same inputs. In the simulator, the stochastic component is mainly confined to bottom-up demand generation: each session is associated with a *seed* and, starting from it, specific seeds for each consumer and/or module are derived deterministically. Consequently, once configuration and parameters are fixed, the generated curves and the results coincide.

Traceability. Traceability concerns the possibility of comparing different executions without losing history. For this reason, the balancing and economics modules record each execution in an index updated in *append-only* mode[7] and maintain a reference to the selected *active run*. Moreover, the economic evaluation saves a *snapshot* of the assumptions (tariffs, incentives, costs, financial parameters) used in the evaluation, so that results remain verifiable and comparable even if, later, the input tables are updated.

In summary, the combination of (i) persistence of *session artefacts*, (ii) run indexing and (iii) control of stochasticity makes it possible to obtain a **reproducible** and **traceable** pipeline, suitable for iterative experimentation on scenarios and configurations.

[7] *Append-only* means that new executions are appended to the history without overwriting previous ones.

Chapter 2

Simulator architecture

This chapter provides an operational description of the simulator architecture and of the end-to-end application flow[1]. The objective is not to discuss in detail the physical and economic models, which are addressed in the subsequent chapters, but rather to define precisely how the different components interact, which data are produced and consumed in each phase, and how such data are persisted to guarantee reproducibility and traceability of experiments.

Figure 2.1 is the canonical representation of the workflow[2]. It summarises the sequence of application phases and highlights the integration points between modules, including the "bridge" phase that materialises and transfers energy profiles towards the balancing calculation. The entire chapter is structured as a reading guide for the figure: for each block, responsibilities, prerequisites and filesystem artefacts[3] are clarified, while references to the subsequent chapters guide the in-depth discussion of individual methodologies.

2.1 Software organisation

The simulator is organised according to a clear separation between presentation level and calculation level. The application layer, implemented in Streamlit[4], has the task of orchestrating the workflow phases, acquiring user inputs, managing session state and invoking the calculation modules. The domain logic is instead concentrated in a

[1] *End-to-end* refers to the entire operational flow: from input definition to the production of final results, including all intermediate phases.

[2] In the following, *workflow* refers to the ordered sequence of application phases and data transformations during simulator execution.

[3] *Filesystem* refers to the operating system's file and folder structure; here it is used as support for saving inputs, outputs and intermediate results.

[4] Streamlit is a Python framework for creating interactive web applications; in this thesis it constitutes the interface through which the user sets parameters, starts phases and visualises results.

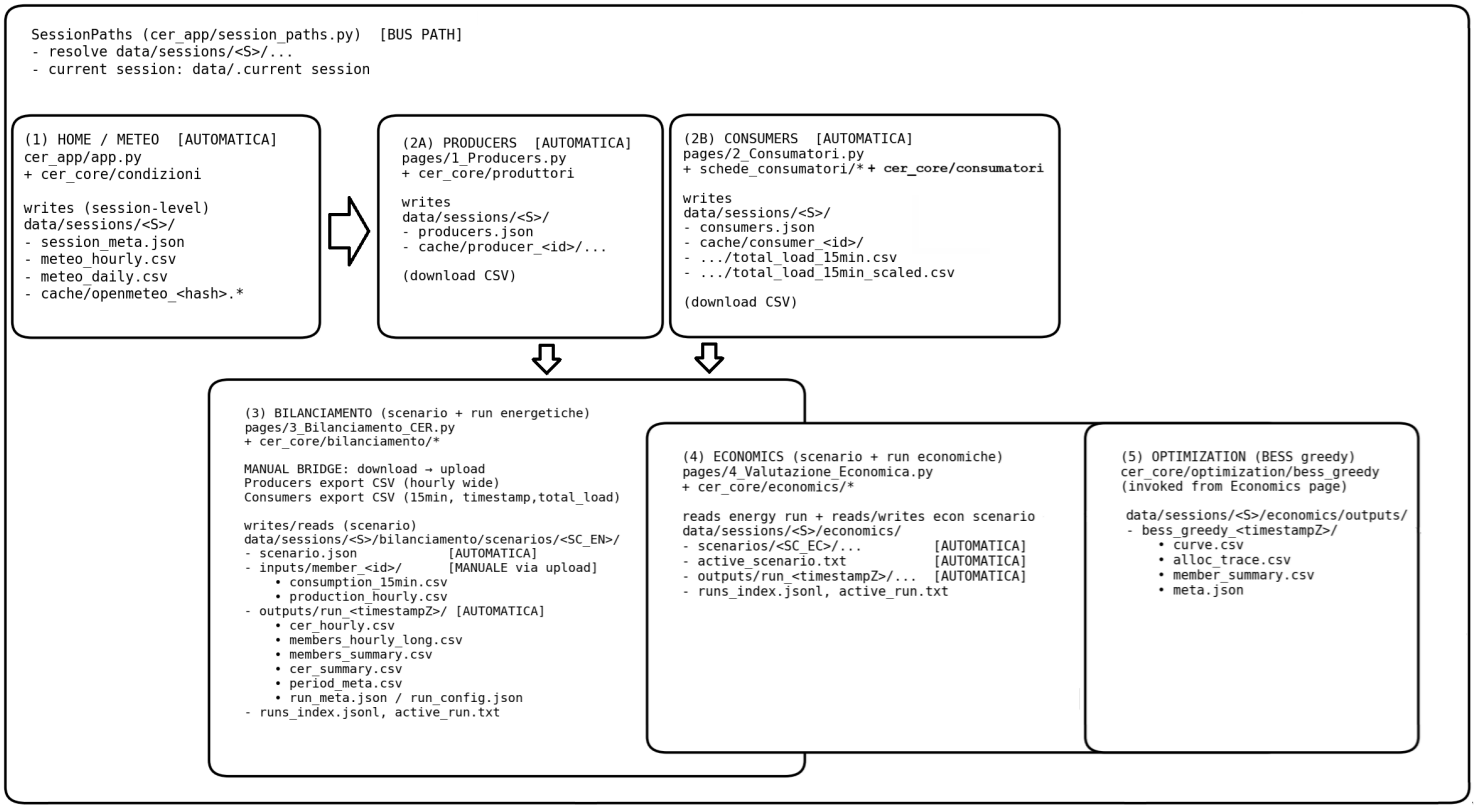

Figure 2.1: Simulator workflow: application phases, integration points and persistence on filesystem (session, scenarios and runs).

core[5] that implements the models and data transformations, with the aim of keeping functions as deterministic and reusable as possible.

From a source-code standpoint, this separation materialises in two macro-modules: `src/cer_app` and `src/cer_core`[6]. The former contains the Streamlit application, the pages corresponding to the main phases, and utilities for path management and the active session. The latter contains the domain subsystems for weather acquisition, generation creation, consumption modelling, energy balancing, economic evaluation and optimisation. In this architecture, the application level governs workflow progression and materialises outputs into persisted artefacts, while the core is responsible for calculation correctness and data-format consistency.

[5]In the following, *core* refers to the software nucleus that contains the calculation logic and data transformations, independent of the user interface.

[6]Project nomenclature: `cer_app` hosts the interface and orchestration; `cer_core` hosts models and calculation functions.

2.2 Persistence, sessions, scenarios and runs

A fundamental design choice of the simulator is the adoption of the filesystem as the main persistence mechanism. This decision makes execution state transparent, facilitates reproducibility of results, and allows a posteriori inspection of inputs, outputs and intermediate artefacts without depending on external services. Persistence is structured on three hierarchical levels, each with a specific semantics: *session*, *scenario* and *run*.

The *session* is the basic work unit. It contains the general metadata of the analysis and the artefacts shared among the different phases, such as meteorological datasets and component configurations. Operationally, a session corresponds to a dedicated directory under `data/sessions/nome_sessione`. The simulator also maintains a pointer to the active session (via a dedicated file), so as to allow resuming work and navigating between sessions.

Within the session, at least the general metadata are persisted (for example in `session_meta.json`[7]), weather datasets (typically in `meteo_hourly.csv`[8] and `meteo_daily.csv`), the producers configuration (for example `producers.json`) and the consumers configuration (for example `consumers.json`). Alongside these artefacts, a cache directory[9] makes it possible to store intermediate results and derived data (for example outputs of external services or profiles for individual components) in order to reduce recalculations and make the interactive experience smoother.

The *scenario* is a logical container used in downstream phases in which it makes sense to repeat processing by varying part of the inputs while keeping the session context constant. In the simulator, the concept is particularly relevant in energy balancing and economic evaluation: a scenario gathers canonical inputs[10] in a stable format independent of the source with which such inputs were generated (for example internally by the simulator or provided by the user). The scenario directory is therefore a "contract" between different phases and ensures that the calculation can be re-executed in a controlled manner.

The *run* is a single execution instance of a scenario. Each run produces a coherent set of outputs and is recorded in an append-only index, with a pointer to the active run shown in the interface. This mechanism makes it possible to compare results from

[7]JSON (*JavaScript Object Notation*) is a structured textual format for data and configurations (key–value pairs), used here for parameters and metadata.

[8]CSV (*Comma-Separated Values*) is a textual tabular format for data exchange: each row represents a record and columns are separated by a delimiter (comma or semicolon).

[9]*Cache* refers to the storage of reusable intermediate results, to avoid recalculations and make the application more responsive.

[10]In the following, *canonical inputs* refers to a standardised representation (fixed schema and conventions) used as a stable interface between different phases of the simulator.

successive runs, preserve the processing history and avoid overwriting outputs. The same logic is adopted both for energy balancing results and for economic evaluation results.

2.3 Workflow description

Figure 2.1 organises the workflow as a sequence of phases that progressively transform user inputs and context datasets into energy and economic results. The flow highlights two parallel branches, dedicated respectively to generation creation and to consumption modelling, which converge at an integration point before balancing. This convergence is implemented through a "bridge" that materialises the profiles in the canonical scenario format, so as to decouple profile generation from the community calculation.

2.3.1 Path resolution and session initialisation

Each workflow phase reads and writes artefacts in deterministic session locations. For this reason, at startup the simulator resolves the active session and constructs a map of application paths, which is then used by the Streamlit pages and by the core modules to access files consistently. This component, highlighted in the figure as *SessionPaths*, is the basis that makes execution reproducible: given the same session and canonical inputs, the set of produced artefacts is inspectable and, if needed, reconstructable.

2.3.2 Home and weather

The initial workflow phase, indicated as *Home/Meteo*, defines the context of the analysis and produces the basic session artefacts. In this phase the time interval and geographical location are fixed, and the meteorological dataset required by the subsequent phases is acquired. The result is persisted in the session as meteorological time series, typically at hourly resolution, together with the metadata describing the choices made (for example selected coordinates and initialisation parameters). Details of the acquisition procedure and of time conventions are discussed in Chapter 3, to which the reader is referred for further detail.

2.3.3 Producers

The *Producers* phase builds and configures the energy generation sources of the community. Operationally, it acquires or derives plant parameters (for example

geometries, orientations and nominal characteristics) and generates hourly production profiles consistent with the session period. Configuration is persisted in the session, while intermediate results and responses from external services are saved in cache to avoid recalculations. The choice to treat producers as an independent workflow branch makes it possible to iterate on plant configuration without having to rerun demand modelling. The implementation and modelling assumptions are discussed in Chapter 4.

2.3.4 Consumers

The *Consumers* phase models the members' electricity demand. Starting from the session context and weather datasets, load profiles are generated, typically at 15-minute resolution, in order to represent intra-hour dynamics and temperature-dependent components in greater detail. Also in this case, the consumers configuration is persisted in the session, while profiles and intermediate components can be saved in cache per member or per subsystem. Separation from the producers branch makes it possible to iterate on behavioural parameters and demand assumptions while keeping generation unchanged. Details of the consumption models and seed management are discussed in Chapter 5.

2.3.5 Integration bridge and materialisation of inputs

The convergence point between the producers and consumers branches, indicated in the figure as *Manual Bridge*, has a precise architectural function: to transform the generated profiles (or alternatively those provided by the user) into canonical inputs for the balancing scenario. In the simulator, producers and consumers can produce outputs in different forms and with specific conventions; balancing, by contrast, requires a minimum set of tabular files for each member, with defined schema and time granularity. Materialisation in canonical format makes this interface explicit and decouples the phases, enabling both the use of internally generated profiles and replacement with external profiles without modifying the balancing logic.

In operational terms, the bridge consists in exporting consumption profiles (15 minutes) and production profiles (hourly) to CSV files and loading them into the input structure of a scenario. This step is deliberately highlighted as manual in the figure, because it represents a clear boundary between profile generation and community calculation, and constitutes the point at which the user can validate, inspect or replace the inputs.

2.3.6 Energy balancing

The *Balancing* phase builds an energy scenario and calculates the community flows. The scenario gathers, for each member, the canonical input files loaded through the bridge: 15-minute consumption profiles and, when present, hourly production profiles. Starting from these inputs, balancing normalises time conventions, aggregates or integrates the series at the required step, and produces energy outputs typically on an hourly basis in terms of energy. The phase therefore generates a set of per-run outputs, which includes per-member results and aggregated community results, as well as metadata and snapshots[11] of the run configuration.

Balancing adopts a versioning mechanism[12]: each execution creates a new run and updates an append-only log, leaving the outputs of previous runs intact; a pointer identifies the active run selected in the interface. This choice makes controlled iteration on inputs and comparison between configurations possible. The formalism and energy quantities handled are discussed in Chapter 6.

2.3.7 Economic evaluation

The *Economics* phase consumes a specific energy run and evaluates the economic performance of the community and of individual members. In this phase an economic scenario is defined, which gathers the assumptions (tariffs, incentives, investment and operating costs), and an economic run that represents the calculation instance executed on a given energy run. Here too, the simulator adopts persistence and versioning: assumptions are saved in the scenario, while results (economic indicators and cash flows) are persisted in a dedicated output folder for the run and recorded in an index. In this way it is possible to rerun the economic calculation by varying assumptions without altering energy results and while keeping the processing history.

The formulation of the indicators and the model setup are discussed in Chapter 7.

2.3.8 Optimisation

The final workflow phase, indicated as *Optimization*, is an application level that uses the economic evaluation as an objective function[13]. In this thesis, optimisation

[11] Coherent copy of the parameters used in a run, saved together with the outputs to make results verifiable and comparable.

[12] Management of successive executions while keeping history: each execution generates a new run without overwriting the previous ones.

[13] Quantity to be minimised or maximised in an optimisation problem (criterion with which alternative solutions are compared).

is aimed at sizing battery energy storage systems (BESS[14]) through an iterative procedure that, given the same economic scenario, evaluates variants of energy configuration and compares impacts on economic indicators. From an architectural standpoint, this phase is a composition of two steps: a recalculation of balancing for the candidate configuration and a new economic evaluation. The optimisation outputs are persisted as additional artefacts associated with the economic analysis, maintaining the same execution traceability logic.

2.4 Conventions and data contracts

For the workflow in Figure 2.1 to be executable robustly, the phases share a set of data conventions. The time convention is discussed in Chapter 3 (Section 3.2) and constitutes a prerequisite for interoperability[15]: timestamps are treated in UTC and derived datasets maintain timezone consistency along the entire pipeline.

A second element is the management of time resolutions. In the simulator, hourly series (weather and generation) and 15-minute series (consumption) coexist; balancing typically produces hourly results in terms of energy, obtained by integrating power over the time step and aggregating when necessary. This scheme makes it possible to model demand with intra-hour detail while keeping a community balance expressed on an hourly basis, consistent with the quantities typically used for reporting.

Finally, the integration bridge defines the minimum contract for scenario input files. The fact that such files are tabular and persisted in a deterministic directory structure makes the boundary between phases explicit and allows both data audit[16] and, if necessary, replacement of inputs with external data. In other words, the session is not only a container of application state, but also a unit of experimental documentation: it preserves the data needed to reconstruct the path that leads from the initial parameters to the energy and economic results.

2.5 References to subsequent chapters

The architectural description provided in this chapter is intentionally cross-cutting and flow-oriented. Details of weather acquisition and of building the session context are

[14] *Battery Energy Storage System*: electrical energy storage system (batteries) that stores energy and releases it subsequently.

[15] Ability of different modules or phases to exchange data without ambiguity thanks to shared formats and conventions.

[16] A posteriori inspection of inputs, outputs and processing steps to reconstruct how a result is obtained.

addressed in Chapter 3. Production modelling and the related generation modules are discussed in Chapter 4, while the construction of demand profiles and seed management are discussed in Chapter 5. Energy balancing, with the formalisation of flows and community metrics, is developed in Chapter 6. Economic evaluation and the optimisation procedure based on economic indicators are finally discussed in Chapter 7. In this way, Figure 2.1 remains the common reference that connects the detailed analyses, ensuring consistency between implementation and narrative.

Chapter 3

General conditions: global configuration and climatic component

This chapter describes how the simulator defines and materialises the *general conditions* of a simulation session. This part is central because it builds the **baseline data repository** (datasets and metadata) on which all downstream modules rely: once the **time interval**, **geographical location** and **meteorological dataset** are fixed, every subsequent analysis (generation, consumption, balancing and evaluation) consistently inherits the same inputs, ensuring reproducibility and traceability.

In the simulator, the general conditions are not limited to external temperature alone: an hourly multivariate weather dataset is acquired and persisted that includes: (i) temperature at 2 m above ground, (ii) wind speed at 100 m, (iii) wind direction at 100 m. These variables support both demand modelling (for example, loads sensitive to climatic conditions) and generation modules (for example, wind power), without requiring downstream modules to query external services.

From an implementation standpoint, the general conditions derive from the cooperation between:

- **application layer** (`cer_app/app.py`), responsible for sessions, user interaction and on-disk persistence;
- **core layer** (`cer_core/condizioni_generali/condizioni_climatiche.py`), responsible for calls to Open-Meteo[1] [6] and for building the weather dataset.

[1] Open-Meteo is a service that provides meteorological data via APIs; in the simulator it is used to obtain multivariate hourly series from geographical coordinates and from a time interval.

3.1 Simulation session: metadata and persistence

Each scenario is represented by a *session*, identified by a name and associated with a dedicated directory:

`data/sessions/{nome_sessione}/`.

The simulator maintains a pointer to the current session in the file `data/.current_session`. If the pointer is not present, the application initialises a default session, so as to always have a defined operating context.

Disk path logic is standardised in the module `cer_app/session_paths.py`. In particular:

- `set_current_session(name)` sets the current session and creates the minimal on-disk structure;
- `get_current_session_dir()` retrieves (or initialises) the directory of the current session;
- `get_paths(...)` builds the session path conventions and creates the minimal required folders.

It is important to note that this module **does not** decide the contents of the files: it only defines *path conventions* used consistently by the application.

Within the session directory, the file `session_meta.json` collects the global metadata:

- `start`, `end`: simulation start and end dates;
- `seed`: numeric seed to control any stochastic components;
- `location`: object that represents the geographical location.

Location is not handled as a simple string (municipality name), but at least as the pair (lat, lon). This choice reduces ambiguity (homonymy or typing errors) and makes requests to external services deterministic: with the same coordinates and time interval, the acquisition process is repeatable.

3.2 Choice of the time interval and UTC convention

The simulator uses UTC as the end-to-end time reference for all persisted datasets. Requests to Open-Meteo explicitly specify `"UTC"` and timestamps are handled as

indices with time-zone information. This choice eliminates implicit conversions linked to local time and simplifies the management of changes between standard time and daylight saving time, which in local time zones may produce duplicate or missing hours.

The reference granularity of climatic conditions is **hourly**. The weather dataset is therefore represented on a sequence of instants $\mathcal{T}$ at constant step of 1 h, with full coverage:

$$\texttt{start 00:00 UTC} \rightarrow \texttt{end 23:00 UTC} \quad \text{(inclusive).}$$

To make the analysis robust and seasonally comparable, this thesis adopts as the recommended choice a **full annual interval**, from **1 January** to **31 December** (always in UTC). In this way the session data repository includes the entire seasonal variability and reduces the risk of bias due to non-representative sub-intervals.

3.3 Locality selection via the Geocoding API

Locality selection occurs via the Geocoding API[2] of Open-Meteo. In the interface, the user enters a search string (for example the name of a municipality); when the string exceeds a minimum length, the orchestration function calls the core via `search_comuni_open_meteo()` and receives a list of candidates. For each candidate, name, administrative area and coordinates are available.

Selection is not automatic: the user explicitly confirms the correct candidate. Following selection, `app.py` updates `session_meta.json` by saving both the search string and the `location` object with latitude and longitude, which becomes the sole reference used for weather acquisition. To improve interface responsiveness, search results can be temporarily cached: this optimisation reduces perceived latency, without altering the simulation logic.

3.4 Acquisition of hourly weather data via Open-Meteo

Generation of the session meteorological dataset is orchestrated at the application layer by the function `ensure_meteo(meta, force)` in `app.py`. The behaviour is deterministic with respect to the session metadata:

[2] *Geocoding* service: starting from a search string (toponym) it returns possible matches with geographical coordinates (latitude/longitude) and administrative metadata, reducing ambiguity and homonymy.

1. the application checks for the presence in the session folder of the files `meteo_hourly.csv` and `meteo_daily.csv`;

2. if the files exist and forced regeneration is not requested (`force = False`), they are reused;

3. otherwise, the application delegates to the core the construction/update of the datasets, using `location`, `start` and `end` present in `session_meta.json`.

This strategy implements session-level caching: downstream modules always consume persisted files, without repeating API calls.

In the core, the entry point for building the multivariate hourly weather dataset is `build_meteo_hourly_for_location_open_meteo(...)` in `cer_core/condizioni_generali/cond`
The builder produces a dataframe[3] with:

- UTC time index, step 1 h;

- full coverage from `start 00:00` to `end 23:00` (UTC, inclusive);

- requested variables with uniform column conventions, including temperature (`temp`, in °C), wind speed (`wind_speed_100m`, in $\mathrm{m\,s^{-1}}$) and wind direction (`wind_direction_100m`, in degrees).

To reliably cover both the historical part and the recent/future tail, the core combines two Open-Meteo endpoints [6]:

- **Archive API**[4] for the historical part;

- **Forecast API** for the most recent portion and for the near future.

Separation occurs with respect to a cutoff[5] defined as a fixed number of days before the current date (archive lag): hours prior to the cutoff are requested from the Archive API, while hours after it are requested from the Forecast API. The Forecast API has a maximum forecast horizon: if the simulation end date exceeds such limit, the core raises an explicit error and requires reducing `end`, avoiding producing an incomplete or heterogeneous dataset.

[3]By *dataframe* we mean a tabular data structure (Pandas) composed of indexed rows (here by timestamp) and columns corresponding to variables.

[4]In the Open-Meteo context, the *Archive API* provides historical data, while the *Forecast API* provides forecast data and the most recent portion; the two endpoints also differ in terms of temporal coverage limits.

[5]Temporal threshold that separates the part requested from the Archive API from the part requested from the Forecast API; the *archive lag* is the margin (in days) with respect to the current date used to define such threshold.

3.5 Session weather outputs and feature engineering

At the end of the pipeline, the data are saved in the session directory. The main files are:

- `meteo_hourly.csv`: hourly series with timestamp in UTC and meteorological variables; it is the shared reference for downstream modules;
- `meteo_daily.csv`: daily aggregation built from the hourly series, useful for synthetic statistics and for logic operating at daily granularity.

In the daily file, for each day (defined with respect to daily boundaries in UTC) descriptive statistics of temperature (mean, minimum, maximum) are computed and, similarly, synthetic statistics for the wind component (for example, means and extreme values of speed). In addition, a qualitative classification of the day based on mean temperature is introduced: `hot` if $T_{\text{mean}} > 25\,°\text{C}$, `cold` if $T_{\text{mean}} < 5\,°\text{C}$, `mild` otherwise. This label is not intended to physically model local meteorology, but represents a simple form of feature engineering[6] useful to introduce discrete climatic context into simplified logic.

3.6 Disk file organisation (session audit)

The session is designed to be inspectable: all inputs and outputs are persisted under `data/sessions/{nome_sessione}/`. In particular, the general conditions use:

- `session_meta.json`: global metadata (`start`, `end`, `seed`, `location`);
- `meteo_hourly.csv`: hourly weather dataset (UTC, 1 h);
- `meteo_daily.csv`: daily aggregated weather dataset;
- `cache/`: on-disk caching directory used by builders to avoid repeated requests with identical parameters (lat/lon/date/variables).

This organisation makes the session a self-contained unit: given the session directory, it is possible to reconstruct datasets and parameters without depending on the interface state.

[6] Construction of derived variables (*features*) from raw data, with the objective of making the dataset more usable in models or decision rules.

3.7 Visualisation and data inspection

To document the climatic component and verify the correctness of the generated data repository, the following reports two representative outputs: (i) an hourly temperature profile over an entire year (seasonal consistency check), (ii) an excerpt of the hourly dataframe, useful to verify structure, columns and units.

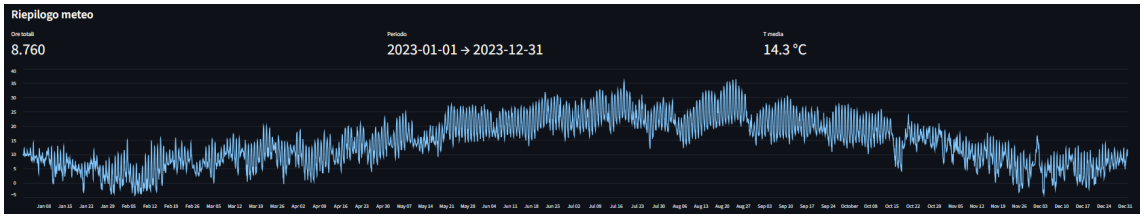

Figure 3.1: Hourly profile of external temperature in 2023 for Doberdò del Lago, generated by the simulator from Open-Meteo data [6].

	timestamp	temp_mean	temp_min	temp_max	wind_speed_mean	wind_speed_min	wind_speed_max	wind_direction_mean	condition
0	2023-01-01 00:00:00+00:00	10.2708	9.4	11.9	1.7442	0.5	2.72	163.4678	mild
1	2023-01-02 00:00:00+00:00	10.0458	9.4	11.4	1.4638	0.38	3.42	248.3048	mild
2	2023-01-03 00:00:00+00:00	10.0792	7.8	12.2	0.7583	0.1	1.48	174.0717	mild
3	2023-01-04 00:00:00+00:00	9.3708	7.6	14.1	1.9138	0.4	3.44	136.4058	mild
4	2023-01-05 00:00:00+00:00	8.8458	6.9	9.7	2.5329	1.77	3.34	168.6367	mild
5	2023-01-06 00:00:00+00:00	8.2542	3.3	12.1	1.7771	0.1	3.6	135.4958	mild
6	2023-01-07 00:00:00+00:00	8.6	7.4	10.6	1.7275	0.5	3.33	173.3289	mild
7	2023-01-08 00:00:00+00:00	8.4208	5.8	11.1	2.5879	0.14	6.58	99.0061	mild
8	2023-01-09 00:00:00+00:00	9.825	6.4	12.1	4.7071	1.12	9.24	95.8889	mild
9	2023-01-10 00:00:00+00:00	7.0625	3.2	12.7	2.2896	0.5	6.02	267.5201	mild

Figure 3.2: Excerpt (first 10 rows) of the session hourly dataframe with timestamp in UTC, temperature and wind variables. The objective is to show the structure of the dataset used by the simulator and the columns made available to downstream modules.

Chapter 4

Energy generation

4.1 Purpose of the chapter and framing within the pipeline

Generation estimation is a central step in the Renewable Energy Communities (CER) simulator, because it determines the temporal availability of renewable energy that can be self-consumed and/or shared, and therefore influences the indicators computed in the subsequent modules. In the developed project, generation is modelled through two technologies: photovoltaic and wind. The implementation choice is aimed at generating plausible, reproducible *hourly* profiles consistent with the time interval and the location defined at *session* level.

From an architectural standpoint, the simulator clearly separates *UI orchestration* and *domain logic*. The `cer_app` component (Streamlit) manages user interaction, producer configuration and persistence of session artefacts. The `cer_core` component instead encapsulates the calculation (geometric estimation, PVGIS querying, conversion from weather to wind power) and the adopted conventions.

Time and unit conventions (module contract). In the simulator, generation time series are treated as **hourly power** and indexed in **UTC**. In particular:

- for photovoltaic, PVGIS returns power `P` in **W** on an hourly basis [7], [8], [9];
- for wind, power is computed internally in **kW** (starting from the power curve in kW) and, when required, can be converted to step energy (kWh) through integration over Δt (in hours).

When conversion from power to energy on an hourly grid is required, the following relation holds:

$$E_h[\mathrm{kWh}] = \frac{P_h[\mathrm{W}]}{1000} \cdot \Delta t\,[\mathrm{h}], \qquad \Delta t = 1\,\mathrm{h}. \tag{4.1}$$

4.2 Session, shared inputs and on-disk artefacts

The simulator is organised around the concept of *session*, which defines location, time interval and associated meteorological dataset. This chapter assumes that the session has already been initialised on the *Home* page and that the session files are available on the filesystem.

4.2.1 Files relevant to generation

All configuration and generation results are *session-scoped*, i.e., saved under the directory of the current session `data/sessions/<session_name>/`. In particular:

- `meteo_hourly.csv`: hourly weather dataset indexed in UTC, obtained via Open-Meteo [6]. It is the *source of truth* for wind generation.
- `producers.json`: configuration of producers and areas (geometries, orientation, tilt, PV parameters, wind parameters). It is the *source of truth* for producer-side configuration.
- `cache/producer_<id>/...`: per-producer cache, used to persist processing outputs (PVGIS and wind) and avoid recalculations when the configuration does not change.

4.2.2 Module responsibilities: UI vs core

This chapter refers to the following modules (as implemented in the ZIP):

- **UI (Streamlit)**: `cer_app/pages/1_Producers.py` orchestrates the producer creation/editing flow and invokes domain functions.
- **PV core**: `cer_core/produttori/produttori.py` implements `producers.json` persistence, angle conventions, PVGIS calls and PVGIS caching.
- **Geometry core**: `cer_core/produttori/laf_packing.py` implements panel

packing[1] in the *Local Area Frame* (LAF)[2]; `cer_core/produttori/roof_3d.py` produces a 3D *mesh*[3] for visualisation.

- **Wind core**: `cer_core/produttori/eolico.py` computes hourly generation from `meteo_hourly.csv` and a power curve (from library or override) and manages a dedicated cache.

4.3 Workflow of the `Producers` page and configuration persistence

The `Producers` page is the UI orchestrator for generation. Operationally, the flow is designed to make the link *user input* → *persisted configuration* → *generated hourly series* explicit.

4.3.1 Producer creation/editing (CRUD)

The UI enables create, read, update and delete operations on producers (CRUD)[4]. Each change is persisted in the session via the core utility functions:

- `load_producers(session_dir)`: loads `producers.json` (list of producers), with robust handling of missing or corrupted files;
- `save_producers(session_dir,producers)`: writes `producers.json` atomically (tmp + replace).

4.3.2 Definition of PV areas and geometric conventions

For each producer the user draws one or more roof areas on the map (GeoJSON polygons[5]). Geometries are stored in `producers.json` in geographic coordinates

[1] In the geometry context, the term *packing* indicates a placement problem: elements (here rectangular panels) are placed within an area while respecting geometric constraints (dimensions, spacing, margins), in order to estimate an admissible layout and the installable count.

[2] *Local Area Frame* (LAF) denotes a local, planar reference system (in metres) associated with the roof area, useful for performing geometric and metric operations (e.g., distances, areas and placement) while avoiding the distortions of geographical coordinates.

[3] By 3D *mesh* we mean a discrete representation of a surface through a set of polygons (typically triangles), used here for visualising roof geometry and installed elements.

[4] Acronym for *Create, Read, Update, Delete*: set of typical operations for managing entities (here producers) in an application, respectively creation, reading, updating and deletion.

[5] Standard JSON-based format to represent geographic geometries (points, lines, polygons) and related attributes; in the simulator it is used to describe roof areas in an interoperable manner.

(EPSG:4326[6]) and, when metric operations are needed (e.g., packing), are projected into a local UTM[7] determined automatically (function `auto_utm_crs`).

Each area also includes:

- a discrete **orientation class** (`N, NE, E, SE, S, SW, W, NW`);
- a **true angle** `angle_deg` in degrees according to the internal convention **0=S, 90=E, 180=N, 270=W**;
- a **tilt** `tilt_deg` in degrees.

These choices are not only descriptive: they become input parameters of the PV model and are also used for conversion to the convention required by PVGIS (Section 4.4.2).

4.4 Photovoltaic generation

Photovoltaic generation is obtained by querying PVGIS (*seriescalc* endpoint), which returns an hourly power series as a function of location and plant parameters [7], [8], [9]. In the simulator, orchestration (UI, area selection, parameters) is separated from calculation logic (kWp estimation and PVGIS call).

4.4.1 Estimation of installable capacity: packing in the Local Area Frame

Once the areas are defined, the simulator estimates the number of installable modules through a *packing* algorithm operating in a local metric system (LAF), so as to work in metres rather than in geographic degrees. The function invoked by the UI is `pack_panels_laf_lonlat`, which:

- projects the area into a local UTM;
- places rectangular panels while respecting panel dimensions and spacing parameters (gap, margin);
- returns the panel footprints (polygons) and counts (`n_panels`).

To ensure reproducibility and ease of rendering, panel footprints can be saved directly in `producers.json` (field `panels_geojson`).

[6] EPSG code of the WGS84 reference system: latitude and longitude expressed in degrees.

[7] *Universal Transverse Mercator*: family of map projections that express coordinates in metres (by zone), useful for metric calculations on distances and areas.

4.4.2 PVGIS querying and angle convention

For each selected area, an hourly power profile is requested from PVGIS. In the core, this operation is encapsulated in `pvgis_series_hourly`, which returns a `DataFrame` indexed in UTC with column `P` (W).

A critical point for model consistency concerns angle conversion. The simulator uses azimuth with the convention:

$$0^\circ = \text{Sud},\ 90^\circ = \text{Est},\ 180^\circ = \text{Nord},\ 270^\circ = \text{Ovest}.$$

PVGIS instead uses the parameter `aspect` with 0=S, positive values towards West and negative values towards East. Conversion is handled explicitly by the function `pvgis_aspect_from_project_azimuth` before the API call, avoiding ambiguity in orientation definition.

4.4.3 PV generation caching

To avoid repeated PVGIS calls when the configuration does not vary, the simulator implements a per-producer cache under `<session>/cache/producer_<id>/pvgis/`. The UI uses:

- `producer_cache_dir(session_dir,producer_id)` to resolve and create the producer cache directory;
- `cache_pvgis_load` / `cache_pvgis_save` to read/write PVGIS request results.

In this way, for identical inputs (geometry, tilt, azimuth, kWp, losses, technology and PVGIS year), the hourly profile is reused without contacting the external service again.

4.4.4 Inputs of the photovoltaic model

To make assumptions transparent and support reproducibility, Table 4.1 summarises the main inputs used in generating photovoltaic production.

4.5 Wind generation

Unlike photovoltaic, wind generation is computed *internally* starting from session weather data. The module `cer_core/produttori/eolico.py` explicitly states that it **does not download external weather data**: the *source of truth* is the file `meteo_hourly.csv`, generated in session via Open-Meteo [6].

Table 4.1: Main inputs of the photovoltaic model in the simulator. Configuration is defined in the interface and persisted in `producers.json`; the hourly series is obtained via PVGIS [7], [8].

Category	**Input**
Area geometry	Area polygon (GeoJSON, EPSG:4326); metric projection (UTM) for packing and area calculation; representative point for PVGIS querying.
Orientation	Tilt and azimuth (`angle_deg`) according to internal convention 0=S, 90=E, 180=N, 270=W; conversion to `aspect` for PVGIS.
Panel packing	Panel dimensions (`panel_w`, `panel_h`); `gap` and `margin`; coverage criteria (`coverage_min_pct`); optional panel footprint in `panels_geojson`.
Peak power	Estimated panel count (`n_panels`); nominal power per panel (`wpp`); total power in kWp per area and aggregable at producer level.
System assumptions	Global losses (`system_loss`, %); PV technology (`tech`); reference year(s) selected in the interface.
Output	Hourly power series `P` (W) indexed in UTC; caching in `cache/producer_<id>/pvgis/`.

4.5.1 Technical pipeline: weather → hub-height speed → power curve

The calculation follows a performance-based pipeline (in line with engineering approaches based on power curves) [10], [11]:

1. reading the hourly weather data and selecting the wind-speed column (e.g., `wind_speed_100m`);

2. extrapolating speed to hub height via a power law (shear):

$$V_{\text{hub}} = V_{\text{ref}} \left(\frac{H_{\text{hub}}}{H_{\text{ref}}} \right)^{\alpha}, \tag{4.2}$$

   where H_{ref} is the weather reference height, H_{hub} the hub height and α the configurable exponent [10], [12];

3. speed→power conversion via a discrete power curve and linear interpolation;

4. application of global losses (%) to obtain net power;

5. aggregation of N enabled turbines.

Power-curve definition is consistent with the idea of measured and tabulated performance (even without implementing advanced corrections, e.g., air density) and aligns with the rationale of performance standards [13].

4.5.2 Turbine library and curve override

The simulator includes a library of turbine models in `wind_turbine_library.json`. The core loads it with `load_wind_turbine_library`. For each configured turbine, the power curve is resolved with the following priority:

1. `power_curve_override` (if present);
2. `power_curve` (legacy);
3. `model_id` pointing to a model in the library.

4.5.3 Wind generation caching

A per-producer cache is also provided for wind, under `<session>/cache/producer_<id>/eolico/`. The cache key is built so as to correctly invalidate results when relevant inputs change. In particular, the module considers:

- the hash of the contents of `meteo_hourly.csv`;
- a canonical hash of the turbine parameters (JSON);
- the hash of the turbine library;
- an algorithm version.

The UI uses dedicated functions (e.g., `build_eolico_cache_key`, `cache_eolico_load`, `get_or_compute_eolico_hourly`) to load or (if necessary) recompute the hourly series.

4.5.4 Inputs of the wind model

Table 4.2 summarises the inputs of the wind model, distinguishing between session meteorological data and turbine-specific parameters.

Table 4.2: Main inputs of the wind model in the simulator. The weather source is the session dataset generated via Open-Meteo [6]; the speed-to-power conversion follows an approach based on shear and power curves [10], [11], [13].

Category	**Input**
Session weather data	`meteo_hourly.csv` (UTC); wind-speed column (e.g., `wind_speed_100m`); temporal indexing in UTC [6].
Heights and shear	Reference height H_{ref}; hub height H_{hub}; power-law exponent α.
Turbine	Turbine model (`model_id`); discrete power curve (v, p) from library or override; number of enabled turbines.
Losses	Percentage global losses (%); aggregation criteria (per turbine, per area/instance, per producer).
Output	Hourly power series (kW) or step energy (kWh) depending on the selected mode; caching in `cache/producer_<id>/eolico/`.

4.6 Produced outputs and use in subsequent modules

The output of the generation phase consists of one or more hourly series (one per area/instance, aggregable at producer level) saved in cache and retrievable in subsequent steps. At pipeline level:

- **configuration** (geometries, PV/wind parameters) is always retrievable from `producers.json`;
- **hourly series** are retrieved from cache (PVGIS/wind) if already computed;
- subsequent modules (CER balancing, economic evaluation) consume these series as inputs, maintaining temporal alignment in UTC.

4.7 Qualitative checks and model limits

Modelled generation is designed to be verifiable and reproducible from session artefacts. At qualitative level:

- in photovoltaic, zero production is expected during night hours and daytime maxima; differences among areas derive from orientation, tilt and estimated installable capacity (packing);

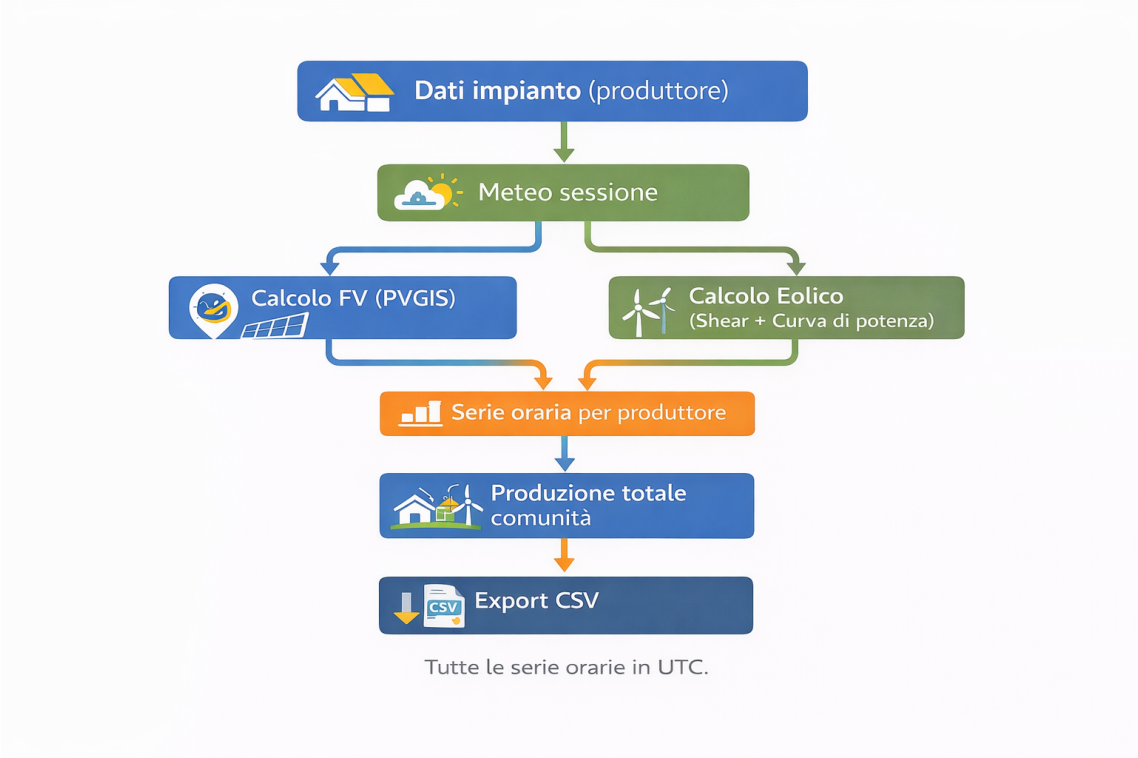

Figure 4.1: Workflow diagram for generating photovoltaic generation in the simulator. Orchestration is handled in `cer_app` (Streamlit UI), while geometric estimation, PVGIS querying and caching are delegated to the domain modules in `cer_core`. Caching logic avoids repeated calls when the configuration does not vary.

- in wind, profile variability directly follows wind dynamics in the session hourly weather data [6]; the parameters H_{hub}, α and losses enable controlled sensitivity analyses.

The chapter describes a methodological compromise consistent with the simulator objective: prioritising transparency of assumptions, reproducibility and temporal consistency over detailed physical models that would require additional inputs (for example complete vertical profiles, air density and wake-effects modelling).

4.8 Figures of the chapter

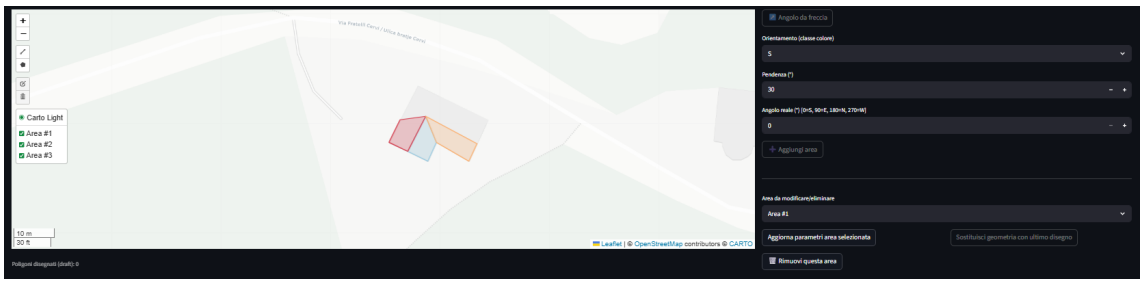

Figure 4.2: User interface for defining photovoltaic areas. The user draws an installable surface and sets, for each area, the essential geometric parameters (tilt and orientation) that drive installable-capacity estimation and the PVGIS request.

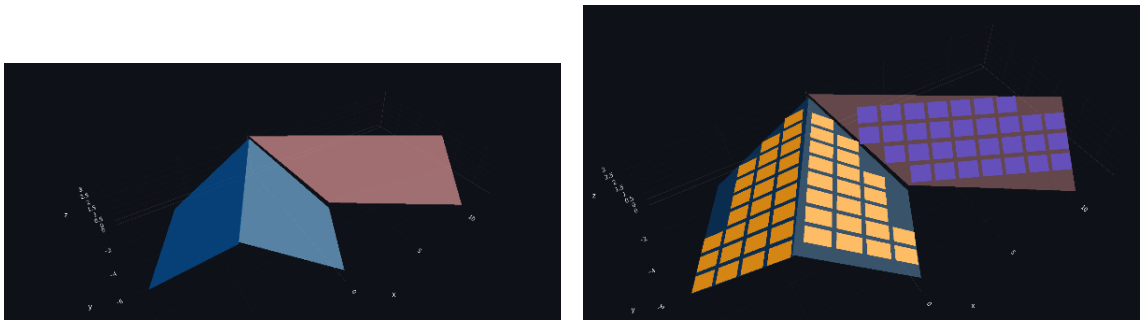

Figure 4.3: 3D render of the roof without panels (left) and with panels positioned according to the packing logic (right). The visualisation supports qualitative validation of installable-capacity estimation, highlighting geometric constraints and spacing.

Chapter 5

Electricity demand: bottom-up modelling of consumption

5.1 Model and integration in the pipeline

5.1.1 Framing and requirements of the demand subsystem

The *demand* subsystem generates, for each consumer, an electric power profile $P(t)$ on a 15-minute time basis, consistent with the characteristics of the meter point (number of residents, appliance stock, habits), with external conditions (weather) and with reproducibility and traceability constraints on the outputs. The adopted setting is *bottom-up*: the total profile is not estimated directly, but emerges from the sum of separately modelled and interpretable components (base loads, occupancy, kitchen, laundry, climate), following approaches consolidated in the literature [14], [15], [16]. At the national level, a parametric and modular modelling is consistent with analyses and reference data on residential consumption [17], [18] (see also [17] and [18]). Moreover, the integration between demand models and the overall energy context is in line with integrated approaches described in the literature [19].

The design goal is not to maximise the physical complexity of each subsystem, but rather to build a profile generator suitable for *what-if*[1] scenarios. The guiding principle is that a variation in the inputs (for instance appliance stock, time constraints, number of weekly cycles, setpoint[2] and system) must translate into an observable and explainable variation of the curve. For this reason, the simulator adopts a decomposition into modules that produce persisted intermediate outputs, so as to enable verification, debugging and reuse along the pipeline.

[1]By *what-if* we mean a scenario analysis in which the user varies, in a controlled manner, some input parameters in order to observe the effect on the resulting profile.

[2]Target value (e.g., indoor temperature) that a controller attempts to maintain.

5.1.2 Modelling rationale: why this implementation

A top-down approach, based on category average curves or on aggregated regressions, can be effective when robust calibrations on external datasets are available and the objective is to reproduce average statistics. In the context of the CER simulator, by contrast, the dominant requirement is the traceability of the link between scenario choices and the temporal shape of demand. Bottom-up decomposition makes it possible to distinguish the quasi-stationary component (base loads) from behavioural contributions (occupancy, kitchen, laundry) and from thermo-dependent ones (climate), preventing the profile from being an "opaque object". This compromise is consistent with the literature on high-resolution residential profile generators based on occupancy, activity and stochasticity [15], [20], [21].

The choice of a 15-minute granularity is functional to energy balancing and to alignment with the quantities of the CER simulation; the presence of hourly weather inputs is handled through realignment onto a 15-minute timebase[3], so that all modules operate on a common and unique time index. Within this framework, modularity is not only a code-organisation choice, but a methodological constraint: each module must have a reduced set of inputs that can be controlled in the UI and must produce a verifiable output, preserving reproducibility and separability of contributions.

5.1.3 Workflow structure and produced artefacts

Figure 5.1 outlines the workflow of the *Consumers* page. The user defines the consumer configuration, the system generates the component curves on a 15-minute basis, saves the artefacts in the dedicated cache and finally produces the total profile, optionally scaled to an annual target. Input persistence and output caching constitute a key point: they make the link between inputs, algorithms and resulting curves explicit and allow the contribution of each module to be inspected independently.

5.1.4 Generated files and output conventions

Table 5.1 summarises the typical outputs produced in the consumer cache (15-minute resolution). The total aggregate and the scaled version (if enabled) are saved respectively in `total_load_15min.csv` and `total_load_15min_scaled.csv`. All files are time series in kW with the same time index, a necessary condition to make the sum of the components a well-defined operation. In the following, the acronym

[3]Temporal resolution/granularity of the index on which the series are discretised and aligned.

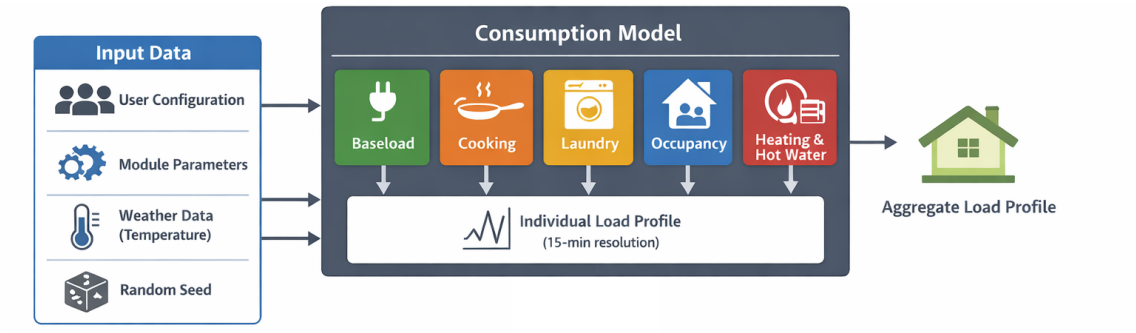

Figure 5.1: Conceptual scheme of the workflow within the *Consumers* page.

ACS[4] denotes domestic hot water, included in the climate module.

Table 5.1: Typical outputs of the consumption subsystem in the per-consumer cache. All files are 15-minute CSVs in kW and share the same time index.

Component	File	Content
Base loads	`baseload.csv`	Always-on, standby and fridge/freezer cycles
Occupancy	`occupancy_15min.csv`	Presence/activity and occupancy-driven loads (e.g., lighting/electronics)
Kitchen	`kitchen.csv`	Events related to meals and use of kitchen appliances
Laundry	`laundry_total.csv`	Washer/dryer cycles allocated within admissible windows
Climate	`climate.csv`	Thermo-dependent load (heating/cooling/ACS)
Total	`total_load_15min.csv`	Sum of the components realigned on a common index
Scaled total	`total_load_15min_scaled.csv`	Total profile multiplied by an energy scaling factor

5.2 Demand generation modules

5.2.1 Base loads: always-on, standby and cycles (Baseload)

Base loads (*baseload*)[5] represent the low-variability component of demand. This category includes *always-on*[6] consumption, *standby*[7] consumption, and devices that

[4]ACS: *Domestic Hot Water*.

[5]Relatively stable demand component with low short-term variability, not directly linked to specific events/activities.

[6]Loads permanently supplied and almost continuous (e.g., routers or electronics operating continuously).

[7]Power absorbed by devices that are inactive but ready for use.

follow a cyclic ON/OFF behaviour[8], such as fridges and freezers. The resulting curve is typically characterised by an almost constant level, with oscillations due to the cycles and with the absence of peaks strongly linked to human activity, which are instead modelled by the behavioural modules. This setting is consistent with the literature on bottom-up profiles and makes it possible to clearly separate the "structural" component of demand from event-driven loads [14], [15].

Listing 5.1: Compact pseudo-code for base-load (baseload) generation.

```
P_const_W = sum(always_on_W) + sum(standby_W)
P_cycle_W = 0
for device in [fridge, freezer_if_present]:
    state = init_state(seed)
    for t in INDEX:
        state = markov_ON_OFF_step(state, seed, t)
        P_cycle_W[t] += (state==ON) * P_device_W
P_baseload_kW = (P_const_W + P_cycle_W) / 1000
save_csv("baseload.csv", P_baseload_kW)
```

In Listing 5.1 the function `markov_ON_OFF_step` implements a Markov chain step[9] to describe the ON/OFF switching of cyclic loads.

5.2.2 Occupancy-driven: presence, activity and lighting

The *occupancy*[10] module builds a temporal representation of presence and domestic activity, starting from a typical week and variability parameters, with the aim of avoiding overly deterministic profiles. In the literature, occupancy is recognised as one of the main drivers of high-resolution residential consumption and constitutes the basis for generating shared loads and for modelling lighting [15], [21]. The lighting component is modelled by combining the presence condition with a proxy[11] of "darkness", consistently with classic approaches that link switching on lights to presence and ambient lighting conditions [22], [23].

Listing 5.2: Compact pseudo-code for occupancy-driven generation.

```
occ_local = weekly_matrix_to_series(matrix, timezone)
occ_local = add_jitter(occ_local, variability, seed)
occ = local_to_UTC(occ_local).reindex(INDEX).ffill()
```

[8]Two-state dynamics (on/off) with switching over time.

[9]Stochastic process in which the probability of the next state depends only on the current state.

[10]Representation (typically discrete) of presence/absence and, more generally, of domestic activity; *occupancy-driven* indicates loads whose activation or power depends on the presence/activity state.

[11]Surrogate variable used as an indirect indicator of a quantity that is not measured directly.

```

for t in INDEX:
    P_small_W[t] = P_min if occ[t]==AWAY else P_min +
        sample_sessions(seed, t)

P_light_W = present(occ) * darkness_proxy(t) * P_lighting_W
save_csv("occupancy_15min.csv", (P_small_W + P_light_W)/1000)
```

In Listing 5.2 the term `add_jitter` introduces a controlled jitter[12]; the `ffill`[13] operator maintains consistency of the time index after realignment.

5.2.3 Kitchen: event-driven model based on meals

Kitchen consumption is described as a sum of events, since the use of appliances is naturally anchored to activities and routines (meal preparation, cleaning). The *event-driven*[14] approach makes it possible to generate peaks concentrated in time windows compatible with lunch and dinner and to reproduce longer and more regular cycles for appliances such as dishwashers. This framework is consistent with the use of activity/*time-use*[15] based models in the generation of residential profiles [15], [20]. For efficiency aspects and energy classification, reference is also made to the applicable technical regulations, which justify the attention to classes and typical per-cycle consumption [24], [25].

Listing 5.3: Compact pseudo-code for generating kitchen consumption (event-driven).

```
events = []
for day in period:
    meals = sample_meals(meal_matrix, seed, day)
    for meal in meals:
        events += maybe_event("oven", meal.time, params, seed)
        events += maybe_event("induction", meal.time, params,
            seed)
        events += maybe_event("microwave", meal.time, params,
            seed)
        events += correlated_event("hood", tied_to="cooking",
            seed=seed)
```

[12]Small random perturbation (of times and/or values) used to represent variability and reduce determinism.

[13]*Forward fill*: filling missing values by propagating forward the last valid value.

[14]Setting in which consumption is generated by discrete events (switch-ons/cycles) superimposed over time, rather than by a continuous dynamics.

[15]Approaches that use data or diaries on time use (*time-use surveys*) to infer routines, activities and probabilities of appliance use.

```
    events += maybe_cycle("dishwasher", start_after=meals.last
        , params=params, seed=seed)

P_kitchen_W = superpose(events, INDEX)
save_csv("kitchen.csv", P_kitchen_W/1000)
```

5.2.4 Laundry: discrete cycles with scheduling constraints

Washer and dryer are modelled as discrete cycles that occur a number of times per week depending on the configuration of the meter point. A central element is the presence of *scheduling*[16] constraints, expressed through admissible start windows: this reduces "pure" randomness and makes the profile more consistent with realistic habits and with load-shifting strategies. The resulting curve therefore appears episodic, with power blocks associated with the cycles and with a temporal distribution dominated by the constraints. Also in this case, considerations on efficiency and per-cycle consumption can be motivated by referring to regulations and energy classes [26], [27].

Listing 5.4: Compact pseudo-code for laundry generation with constrained scheduling.

```
n_cycles = cycles_from_week_rate(cycles_per_week, period)
slots = start_matrix_to_allowed_slots(start_matrix)
starts = sample_starts(slots, n_cycles, seed)

P_W = 0
for s in starts:
    P_W += place_cycle(washer_profile, INDEX, start=s)
if dryer_present:
    starts_d = sample_starts(slots_dryer, n_dryer_cycles, seed
        )
    for s in starts_d:
        P_W += place_cycle(dryer_profile, INDEX, start=s)

save_csv("laundry_total.csv", P_W/1000)
```

[16]Temporal allocation of the cycles within admissible start windows, possibly with selection/optimisation criteria or habit constraints.

5.2.5 Climate: compact thermo-electric model and dependence on weather

The climate module introduces the dependence of demand on external conditions and on the characteristics of the building/system. The idea is to translate outdoor temperature and setpoint into a thermal demand, then convert this demand into electric power as a function of generator performance. The use of compact lumped-parameter models is a common choice when the objective is to capture inertia and dependence on weather without resorting to detailed building simulations; the literature on *lumped* and RC models[17] provides the methodological reference [28]. Thermo-electric conversion via COP[18], possibly dependent on outdoor temperature, is consistent with simplified formulations used for heat pumps in applied studies [29].

Listing 5.5: Compact pseudo-code for generating the climate load (thermo-electric).

```
T_out = interpolate_to_15min(meteo_hourly.temperature, INDEX)
T_in = T_init
for t in INDEX:
    Q_loss = (T_in - T_out[t]) / R_env
    Q_ctrl = control(setpoint, schedule, T_in, deadband)
    Q_hvac = Q_loss + Q_ctrl
    COP = performance_map(T_out[t], device_type)
    P_elec_kW[t] = max(0, Q_hvac / COP) / 1000
    T_in = update_RC_state(T_in, Q_hvac, C_th, dt)
save_csv("climate.csv", P_elec_kW)
```

In Listing 5.5 the controller uses a *deadband*[19]; the `performance_map`[20] function represents the dependence of performance on external conditions.

[17] *Lumped*: lumped-parameter models, in which distributed properties (inertia, losses) are aggregated into a few equivalent parameters. RC model: thermo-electric representation as a resistance–capacitance (R–C) network to describe losses and thermal storage.

[18] COP (*Coefficient of Performance*): ratio between delivered thermal power and absorbed electric power of the generator, often a function of operating conditions.

[19] Insensitivity band around the setpoint within which the controller does not apply corrections, in order to avoid frequent switching.

[20] Performance map (tabulated or parametric) that associates operating conditions (e.g., outdoor temperature) with generator performance, for instance in terms of COP.

5.3 Aggregation, scaling and interpretation of results

5.3.1 Aggregation of the total profile

The consumer's total profile is obtained by summing the components computed by the modules, all expressed on the same time basis. Aggregation is not only a mathematical operation, but a methodological step that preserves interpretability: each component is an artefact persisted in cache, and the total profile is the result of their composition. In this way, any anomalies can be traced back to the responsible module and corrected by acting on the inputs of the only subsystem involved.

5.3.2 Scaling to an annual energy target

In many cases it is useful to impose an overall energy constraint, for instance an annual target consumption E_{ann} (kWh), while preserving the temporal shape of the generated profile. In this setting, scaling is implemented as multiplication by a single scalar, computed over the simulated period, so as to preserve patterns and seasonality. Denoting by Δt the duration of the time step in hours and by $P(t)$ the power in kW, the raw energy over the period is $E_{\text{raw}} = \sum_t P(t)\Delta t$. If the simulated period covers N_d days, the target over the period is set equal to $E_{\text{target}} = E_{\text{ann}} \cdot N_d/365$; the scaling factor is therefore $\alpha = E_{\text{target}}/E_{\text{raw}}$ and the scaled curve is $P_{\text{scaled}}(t) = \alpha P(t)$. This mechanism is intentionally simple: it keeps unchanged the temporal structure produced by the modules and makes it possible to calibrate the energy level without introducing additional degrees of freedom.

5.4 Profile examples and qualitative discussion

Figure 5.2 shows an example of base loads in the month of January, highlighting an almost constant component with cyclic variations. Figure 5.3 reports an occupancy-driven profile with modulation linked to presence and lighting. Figures 5.4 and 5.5 show event-driven contributions (kitchen and laundry) with peaks localised within time windows consistent with habits and scheduling constraints. Figure 5.6 highlights the effect of weather dependence on the climate load. Finally, Figure 5.7 shows the aggregated profile, obtained as a composition of the previous components, confirming that the overall shape derives from the superposition of interpretable contributions.

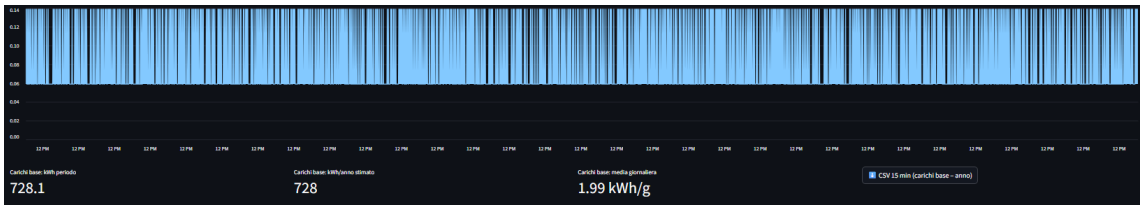

Figure 5.2: Example of base-load profile in the month of January.

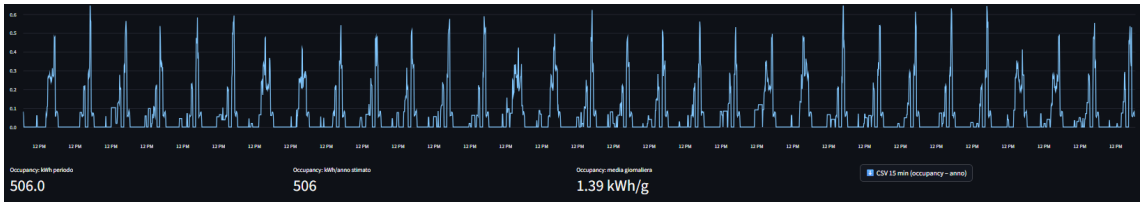

Figure 5.3: Example of occupancy-driven profile in the month of January.

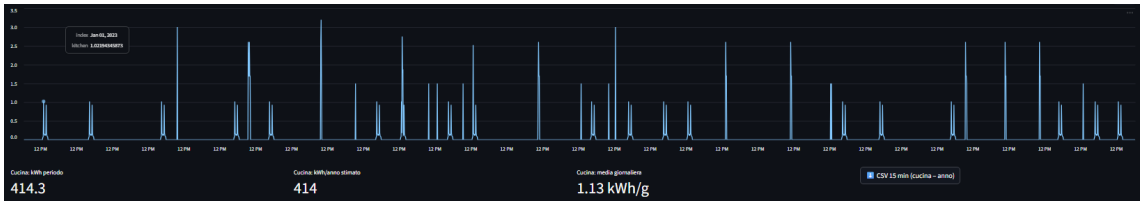

Figure 5.4: Example of kitchen profile (event-driven) in the month of January.

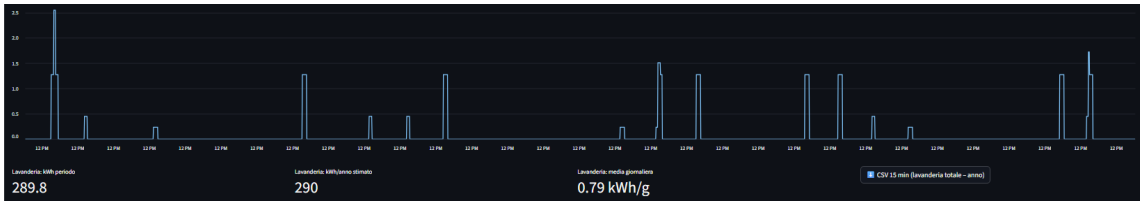

Figure 5.5: Example of laundry profile in the month of January.

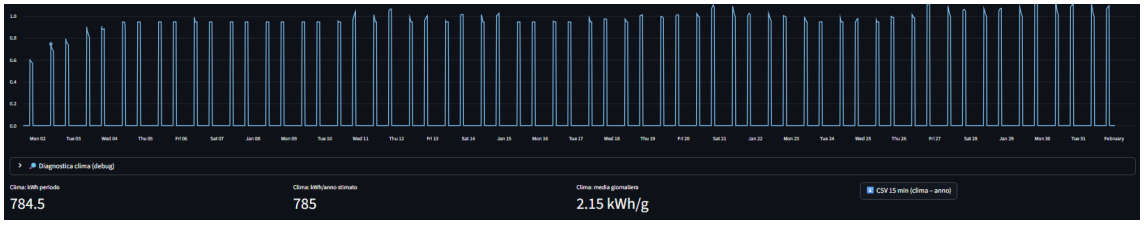

Figure 5.6: Example of climate profile in the month of January.

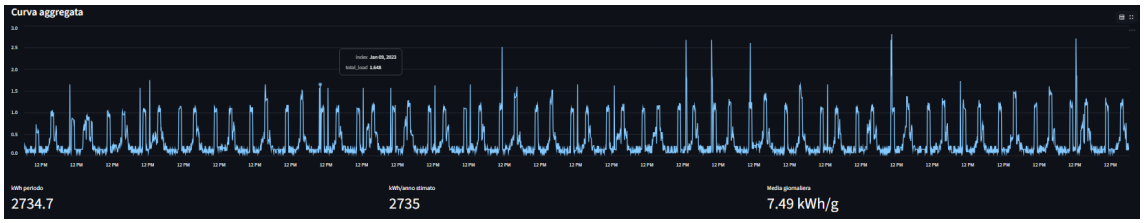

Figure 5.7: Example of aggregated profile in the month of January.

Chapter 6

Energy balancing: implementation, flows and shared energy

The *energy balancing* subsystem constitutes the final stage of the simulation pipeline: downstream of the generation of production curves (Chapter 4) and consumption curves (Chapter 5), it reconstructs the energy flows of individual members and of the collective configuration, with particular attention to hourly shared energy required by the regulation of collective self-consumption [3], [4].

From an implementation standpoint, the dedicated Streamlit page (`3_Bilanciamento_CER.py`) orchestrates the definition and persistence of the *scenario* (member registry, period and simulation options), the loading of consumption/generation profiles and execution of the computation. The computational part is concentrated in the modules in `cer_core/bilanciamento/` (parsing, validation, flow calculation and output production). Each execution produces a versioned *run*, so as to guarantee reproducibility and a deterministic linkage with the economic evaluation (Chapter 7).

6.1 Role of the subsystem and data contract in the pipeline

The objective of balancing is to transform members' power profiles (15-minute consumption and typically hourly generation) into:

hourly energy flows for each member ($E_{\text{prel},i}(h)$, $E_{\text{imm},i}(h)$, any self-consumption and storage components); aggregated community flows ($E_{\text{prel,CER}}(h)$, $E_{\text{imm,CER}}(h)$) and shared energy $E_{\text{cond}}(h)$; finally, energy indicators (KPI) at member and community level..

Results are persisted in tabular files with a stable schema (`members_hourly_long.csv`

and cer_hourly.csv), so as to constitute a *data contract*[1] towards the economic phase: the evaluation monetises withdrawals, injections and, above all, the shared energy $E_{\text{cond}}(h)$.

6.2 Scenario workspace and input persistence

6.2.1 Filesystem structure per scenario and per member

To support comparisons between configurations and experimental repeatability, balancing inputs are organised in a filesystem *scenario workspace*[2]. Each scenario is a standalone folder (e.g. .../bilanciamento/scenarios/<scenario_name>/) that contains:

scenario.json: persisted configuration (members, period, generation/BESS options); inputs/member_<id>/consumption_15min.csv: 15-minute consumption profile (mandatory); inputs/member_<id>/production_hourly.csv: hourly generation profile (optional, for prosumer); outputs/run_<run_id>/: results folder for a single execution (run); finally, runs_index.jsonl[3] and active_run.txt: append-only execution log and pointer to the current run..

This structure clearly separates *input* (scenario and CSV) from *output* (run), preventing the state of the Streamlit interface from becoming a requirement for result reproducibility.

6.2.2 Scenario data model and legacy migration

The scenario.json represents the source of truth for balancing and includes:

list of members (identifier, consumer/prosumer role[4], multiplicity num); generation configuration (mode Totale vs Aree with selection of the Area_* columns); BESS configuration (capacity, DoD, efficiencies, initial SOC) when present; finally, simulation period (if specified) or the parameters needed to infer it..

[1]Data interface between modules: the schema and meaning of the columns are kept stable so that downstream components can consume the outputs without depending on the internal implementation of balancing.

[2]Working directory dedicated to the scenario, which gathers simulation inputs and outputs in a reproducible folder structure.

[3]File in *JSON Lines* (JSONL) format: one JSON record per line; it is used as an append-only registry of runs.

[4]In the following, *consumer* indicates a meter point with consumption only; *prosumer* a meter point that consumes and produces (e.g., via FV) and can therefore also inject energy into the grid.

6.3 Time consistency and data quality

6.3.1 Timezone normalisation and UTC convention

All timestamps are normalised to **UTC** during parsing and persistence. In particular, *naive* timestamps are interpreted in the scenario time zone and then converted to UTC; *timezone-aware*[5] timestamps are first converted to the scenario time zone and finally normalised to UTC. This policy reduces ambiguities related to clock changes (DST) and makes runs and scenarios comparable.

6.3.2 Period definition and construction of the expected indices (T15, TH)

The simulation period can be defined explicitly in the scenario or inferred from the 15-minute consumption data: the simulator computes t_0 as the start of day of the first sample and t_1 as the end of day of the last sample (considering the 15-minute granularity). From $[t_0, t_1)$ the following are constructed:

15-minute index T_{15} for the physical balance (consumption, BESS); moreover, hourly index T_H for aggregation, shared energy and monetisation..

6.3.3 Validations and conservative rules on missing data

The adopted approach is conservative: the series are realigned onto the expected indices and only complete and regular profiles are accepted (monotonicity, absence of duplicates, correct time step). Any *internal* gaps in the series (missing between two expected timestamps) lead to an explicit error, avoiding implicit interpolations that would reduce result traceability.

6.4 Parsing and construction of consumption and generation series

6.4.1 15-minute consumption: format, checks and clipping

By *clipping*[6] we mean trimming the consumption profile to the period $[t_0, t_1)$.

[5] A *naive* timestamp does not include time-zone information; a *timezone-aware* timestamp includes an offset or an IANA zone. DST (*Daylight Saving Time*) denotes the clock change related to daylight saving time.

[6] Trimming the series to the simulation period $[t_0, t_1)$, discarding external samples and ensuring consistency with the expected time indices.

Each member has a 15-minute consumption profile in `consumption_15min.csv` with at least the columns:

`timestamp`	instant (UTC after normalisation)
`total_load`	absorbed power [kW]

The profile is validated and trimmed to the period $[t_0, t_1)$, producing a regular series on T_{15}.

6.4.2 Hourly generation: Totale vs Aree modes and realignment on T15

Generation, when available, is provided on an hourly basis in `production_hourly.csv` with time column `time` and:

mode `Totale`: `Totale` column; moreover, mode `Aree`: sum of the `Area_*` subcolumns selected in the scenario..

The hourly series is validated and realigned to T_H; for the 15-minute physical calculation it is then transformed onto T_{15} via *zero-order hold*[7] (replication of the hourly value across the four 15-minute intervals).

6.5 Member energy balance (without storage)

6.5.1 Notation, units and kW→kWh conversion

In the following, $P_{\text{load},i}(t)$ denotes the load power of member i at step $t \in T_{15}$, while $P_{\text{pv}}(t)$ denotes local generation power (possibly aggregated). Step energies are obtained via:

$$E(t) = P(t)\,\Delta t, \qquad \Delta t = 0.25 \text{ h}. \tag{6.1}$$

6.5.2 Definition of self-consumption, withdrawal and injection

At step t the self-consumed energy is:

$$E_{\text{aut},i}(t) = \min\big(E_{\text{load},i}(t), E_{\text{pv}}(t)\big). \tag{6.2}$$

The grid-exchange components are therefore:

$$E_{\text{prel},i}(t) = E_{\text{load},i}(t) - E_{\text{aut},i}(t), \qquad E_{\text{imm},i}(t) = E_{\text{pv}}(t) - E_{\text{aut},i}(t). \tag{6.3}$$

[7] Resampling technique in which the signal is kept constant between two samples (piecewise-constant interpolation).

6.5.3 Hourly aggregation of flows

Since the shared-energy regulation operates on an hourly basis [3], [4], the flows are aggregated on T_H by summing the 15-minute intervals:

$$E_{\bullet,i}(h) = \sum_{t \in h} E_{\bullet,i}(t), \tag{6.4}$$

where $\bullet$ denotes any energy component (withdrawal, injection, self-consumption, etc.).

6.6 Extension with electrochemical storage (BESS) in RSE-like logic

6.6.1 Parameters, SOC constraints and efficiencies (half-cycle)

The BESS is modelled at 15-minute resolution through state of charge (SOC[8]) expressed in kWh, with capacity C and maximum depth of discharge DoD[9]. SOC limits are:

$$SOC_{\min} = C\,(1 - DoD), \qquad SOC_{\max} = C. \tag{6.5}$$

Given a round-trip efficiency[10] η_{rt}, the model uses a half-cycle efficiency:

$$\eta_{\mathrm{hc}} = \sqrt{\eta_{\mathrm{rt}}}, \tag{6.6}$$

consistent with a symmetric split of losses between the charge and discharge phases.

6.6.2 15-minute simulation step and SOC update

The net terminal energy in the absence of a BESS is defined as:

$$E_{\mathrm{term}}(t) = E_{\mathrm{pv}}(t) - E_{\mathrm{load},i}(t). \tag{6.7}$$

The operational logic follows a local priority: production surplus $\rightarrow$ charge, deficit $\rightarrow$ discharge, residual $\rightarrow$ grid, while respecting the constraints $SOC_{\min}$ and $SOC_{\max}$.

[8] *State of Charge*: available energy in the storage relative to nominal capacity; in this model SOC is expressed in kWh.

[9] *Depth of Discharge*: maximum share of capacity that can be discharged; it determines the lower limit $SOC_{\min}$.

[10] *Round-trip* efficiency measures the overall yield of the charge+discharge cycle; *half-cycle efficiency* represents the yield per single phase (charge or discharge).

The state-of-charge evolution is:

$$SOC(t + \Delta t) = SOC(t) + E_{\text{bess}}(t), \tag{6.8}$$

where $E_{\text{bess}}(t)$ is positive during charge and negative during discharge (in kWh per step).

The grid balance after the battery action becomes:

$$E_{\text{grid},i}(t) = E_{\text{pv}}(t) - E_{\text{load},i}(t) - E_{\text{bess}}(t), \tag{6.9}$$

con:

$$E_{\text{imm},i}(t) = \max\big(0, E_{\text{grid},i}(t)\big), \qquad E_{\text{prel},i}(t) = -\min\big(0, E_{\text{grid},i}(t)\big). \tag{6.10}$$

6.6.3 Discussion of the abstraction level (energy-based) and implications

The implemented model is *energy-based*[11]: the dynamics is expressed in terms of energy exchanged over the step Δt and SOC constraints, without explicitly introducing charge/discharge power limits (kW) or inverter constraints[12]. This choice simplifies scenario-based analysis and maintains pipeline transparency, but may overestimate the BESS ability to track fast peaks: this limitation is considered in the final discussion (Section 6.10).

6.7 Community aggregation and shared energy

6.7.1 Flow summation and definition of hourly shared energy

Denoting by $E_{\text{prel},i}(h)$ and $E_{\text{imm},i}(h)$ the hourly flows of members, the aggregated community flows are:

$$E_{\text{prel,CER}}(h) = \sum_i E_{\text{prel},i}(h), \qquad E_{\text{imm,CER}}(h) = \sum_i E_{\text{imm},i}(h). \tag{6.11}$$

Hourly shared energy is defined as:

$$E_{\text{cond}}(h) = \min\big(E_{\text{prel,CER}}(h), E_{\text{imm,CER}}(h)\big), \tag{6.12}$$

[11]Approach in which the dynamics is modelled in energy (kWh) over the step Δt, with constraints on the state (SOC), rather than through a power model (kW) with dynamic charge/discharge limits.

[12]Constraints imposed by the converter/inverter (e.g., maximum power and current limits) that can reduce the effective charge/discharge capability compared to a purely energy-based model.

consistently with the regulatory definition on an hourly basis [3], [4].

6.7.2 Residuals to the grid (import/export) and consistency checks

Residuals to the grid, once shared energy has been "extracted", are:

$$E_{\text{imp,CER}}(h) = E_{\text{prel,CER}}(h) - E_{\text{cond}}(h), \qquad E_{\text{exp,CER}}(h) = E_{\text{imm,CER}}(h) - E_{\text{cond}}(h). \tag{6.13}$$

By construction, the inequalities $E_{\text{cond}}(h) \leq E_{\text{prel,CER}}(h)$ and $E_{\text{cond}}(h) \leq E_{\text{imm,CER}}(h)$ hold, which are useful as plausibility checks.

6.8 Balancing outputs and run versioning

6.8.1 Output schema (CER, members long, summary, metadata)

Each execution produces structured and reusable outputs:

`cer_hourly.csv`: aggregated hourly profile (includes $E_{\text{cond}}(h)$); `members_hourly_long.csv`: hourly member profiles in long[13] format (key `member_id`); `members_summary.csv`, `cer_summary.csv`: energy KPI; finally, `period_meta.csv`: period and time-normalisation metadata..

These files constitute the minimum output necessary to monetise CER behaviour in Chapter 7.

6.8.2 Append-only registry, active run and configuration snapshot

From a reproducibility standpoint, executing the simulation creates a dedicated folder `outputs/run_<run_id>/` (with `run_id` typically based on a UTC timestamp) containing the outputs and a configuration *snapshot*[14] actually used (e.g. `run_config.json`) as well as run metadata (e.g. `run_meta.json`).

To support traceability across successive executions, the scenario also maintains a registry of energy runs (file `runs_index.jsonl`) and a pointer to the active run (file `active_run.txt`). The registry is updated in append-only mode and associates

[13] *Long* (or *tidy*) representation: one row per (timestamp, member, variable) combination, useful for filtering and aggregations.

[14] "Frozen" copy of the configuration (and, if relevant, of the derived parameters) actually used in a run, so that execution conditions can be reconstructed a posteriori.

to each run its identifier, the paths of the main outputs and a fingerprint[15] of the configuration files. This mechanism makes it possible to deterministically reload the last validated run and compare different runs without depending on the volatile state of the Streamlit interface; moreover, it enables the direct linkage of the economic evaluation to the selected energy run (Chapter 7).

6.9 Targeted comparison with CACER_Simulator (RSE)

The main balancing logics (definition of hourly shared energy and storage dynamics in terms of SOC and constraints) are consistent with the setting of the RSE simulator for CACER [30]. The main difference lies in the architecture: integration into an interactive application enables scenario management, data-quality checks, filesystem persistence and run versioning, making comparative analysis between configurations easier (for example variation of FV power, introduction of the BESS or variations in the mix of meter points).

6.10 Known limitations and extensions

The main limitations (and potential extensions) of the balancing subsystem include:

BESS power constraints: the absence of kW limits may overestimate the action on peaks; a natural extension is to introduce $P_{\max}^{\mathrm{ch}}$ and $P_{\max}^{\mathrm{dis}}$ and inverter constraints.; **Temporal representativeness**: annualisation or the use of partial time windows may introduce seasonal biases; it is useful to support multi-year periods or seasonal weights.

[15]Deterministic imprint (hash) computed from input files/parameters: it makes it possible to verify that configuration and data have not changed between executions.

Chapter 7

Economic evaluation of CER projects and BESS optimisation

This chapter describes the simulator's economic-financial evaluation module and the sizing logic of electrochemical storage (BESS) as a function of financial indicators. The energy balancing subsystem (Chapter 6) returns hourly series of energy exchanges at member and community level; the economic module translates these quantities into monetary counterparts (costs, revenues and incentives) and builds a multi-year cash model, from which typical project KPIs (NPV, IRR and payback) are derived.

Within the simulator, the economic evaluation is intended as a step strictly dependent on the energy run[1]: each evaluation stores a snapshot[2] of the assumptions used (*assumption pack*[3]) and a run log, so as to enable reproducible comparisons across different configurations (FV power, mix of meter points, presence/absence of BESS, tariff and tax parameters).

7.1 Input data and link to the energy run

The economic evaluation operates downstream of a validated energy run. From an implementation standpoint, the dedicated Streamlit page (`4_Valutazione_Economica.py`) loads from the active balancing run two canonical datasets[4]: (i) the community hourly

[1]By "run" we mean a single complete execution of the simulator (energy or economic) with a fixed set of inputs, identified and traced through versioned outputs (e.g. timestamp folder and metadata) to ensure reproducibility.

[2]By "snapshot" we mean an immutable copy (at the time of the run) of the settings and parameters used, saved together with the outputs so that the calculation configuration can be reconstructed ex post.

[3]The *assumption pack* ("package of assumptions") collects, in a structured way, tariffs, price/incentive profiles and tax/technical parameters required for the economic calculation.

[4]In this work, by "canonical datasets" we mean exchange files in a standardised format and naming convention, produced by the balancing module and consumed by the economic module.

profile (`cer_hourly.csv`), containing in particular the shared energy $E_{\text{cond}}(h)$; (ii) the members' hourly profile in long format (`members_hourly_long.csv`), containing for each member i the withdrawal energies $E_{\text{prel},i}(h)$ and injection energies $E_{\text{imm},i}(h)$, in addition to the self-consumption components and any indicators associated with the battery.

At a conceptual level, the data contract between balancing and economics is defined by the following hourly quantities:

$$E_{\text{prel},i}(h), \quad E_{\text{imm},i}(h), \quad E_{\text{aut},i}(h), \quad E_{\text{cond}}(h), \qquad h \in \mathcal{H}, \tag{7.1}$$

In particular, $E_{\text{aut},i}(h)$ represents the energy self-consumed by the member (from local generation and/or from BESS discharge, depending on the configuration) and is used to monetise the bill savings in the cash perspective described in Section 7.5. where $\mathcal{H}$ is the set of simulated hours (UTC timezone, left-inclusive/right-exclusive intervals). The individual quantities derive from the 15 min simulation and subsequent hourly aggregation (Sections 6.5–6.6). The shared energy $E_{\text{cond}}(h)$ is defined as the minimum between the sum of injections and the sum of withdrawals in the community (Section 6.7).

7.2 Economic assumptions and price/incentive profiles

The economic evaluation requires, in addition to the energy flows computed in the technical run (self-consumed, shared and grid-exchanged energy), a set of assumptions that describe the purchase and sale prices of energy and the incentive profiles. In the simulator, these assumptions are managed as an *assumption pack* consisting of: (i) tabular parameters (policies, tariffs, factors) and (ii) optional hourly time profiles ($PZO(h)$), which can be fixed, uploaded by the user or deterministically derived from other variables.

7.2.1 Time profiles: conventions, units and validation

All economic profiles used by the model are hourly series indexed by timestamp and expressed in €/kWh. Operationally, the simulator assumes that the profiles are aligned with the temporal discretisation of the energy run; in particular, an hourly profile must fully cover the simulation time interval. If a required profile is not

They constitute the operational part of the "data contract", i.e. the specification of the variables required (columns, units and temporal conventions) at the interface between the two modules.

available or does not cover the entire interval, the economic run does not proceed, avoiding implicit interpolations that would reduce result traceability.

7.2.2 Hourly zonal price: definition and acquisition modes

In this work the *Prezzo Zonale Orario* (PZO) is treated as an exogenous input of the economic module, represented by an hourly series:

$$PZO(h) \quad [€/\text{kWh}], \tag{7.2}$$

where h denotes the simulation hour.

The simulator does not perform automatic download of PZO: the user can (i) upload a historical/forecast profile obtained from external sources (e.g. exports made available by GME, in compliance with service credentials and terms of use), or (ii) generate a synthetic profile within the interface (random profile reproducible through a seed). This design choice clearly separates the data acquisition chain (outside the model scope) from the economic calculation, fostering reproducibility and auditability.

CSV file format for the PZO profile

The PZO profile is loaded as a CSV file with a time column and a price column. The recommended format is as follows:

- `time`: hourly timestamp (convertible to `datetime`);
- `pzo_eur_kwh`: price value in €/kWh.

Synthetic generation of PZO

As an alternative to uploading, the simulator allows the generation of a synthetic hourly profile $PZO(h)$ via statistical parameters (mean, standard deviation) and a random seed. The resulting profile is non-negative and reproducible given the same parameters and seed; this option is useful for sensitivity analyses and robustness tests of the model.

7.2.3 Energy purchase price

The purchase price of the energy withdrawn from the grid for member i is modelled as an hourly profile $p_i^{buy}(h)$ in €/kWh. The simulator implements three modes.

Fixed price. The price is constant over time:

$$p_i^{buy}(h) = \bar{p}_i^{buy}. \tag{7.3}$$

Three time bands (F1/F2/F3). The price takes three levels as a function of the time band:

$$p_i^{buy}(h) \in \{p_{F1}, p_{F2}, p_{F3}\}, \tag{7.4}$$

where the band assignment is performed according to the hourly classification on the local calendar.

PZO-indexed price with spread. The price is described as an affine function of PZO:

$$p_i^{buy}(h) = m_i^{buy} \cdot PZO(h) + s_i^{buy}, \tag{7.5}$$

where m_i^{buy} is a multiplier (dimensionless) and s_i^{buy} a spread in €/kWh. This option makes it possible to represent parametrically a market-indexed supply with contractual adjustment.

The same profile $p_i^{buy}(h)$ is also used to value, in a DCF perspective[5], the economic benefit of self-consumed energy (bill savings), as described in Sections 7.3 and 7.5.

7.2.4 Energy sale price

The sale price of the energy injected into the grid by member i is modelled as an hourly profile $p_i^{sell}(h)$ in €/kWh. The simulator supports the following modes.

Fixed price.

$$p_i^{sell}(h) = \bar{p}_i^{sell}. \tag{7.6}$$

Price indexed to PZO.

$$p_i^{sell}(h) = m_i^{sell} \cdot PZO(h). \tag{7.7}$$

Price indexed to PZO with fee.

$$p_i^{sell}(h) = m_i^{sell} \cdot PZO(h) - f_i^{sell}, \tag{7.8}$$

[5]DCF (*Discounted Cash Flow*): valuation model that discounts future cash flows at a discount rate r to compare them on an equivalent time-zero basis.

where f_i^{sell} is a fee in €/kWh. This formulation allows representation of imbalance/management costs or contractual withholdings.

7.2.5 TIP incentive profile

The simulator represents the unit incentive (TIP) as an hourly profile $TIP(h)$ in €/kWh. Depending on the selected mode, the profile can be fixed, uploaded or derived from $PZO(h)$.

Fixed TIP.

$$TIP(h) = \overline{TIP}. \tag{7.9}$$

TIP as a simple function of PZO. For parametric analyses, the simulator includes a deterministic mode in which $TIP(h)$ depends on $PZO(h)$ through a base term and an additive term capped by a cap:

$$TIP(h) = TIP_{base} + \min\Big(cap_{max}, \max\big(0, cap_{ref} - PZO(h)\big)\Big). \tag{7.10}$$

This formulation has a modelling purpose (sensitivity to the market price) and is not intended to replace a regulatory determination.

TIP in RSE-like mode. The simulator also implements an "RSE-like" mode consistent with the regulatory references assumed in the chapter. In this mode, the calculation is expressed in €/MWh and then converted to €/kWh. We define:

$$PZO_{MWh}(h) = 1000 \cdot PZO(h). \tag{7.11}$$

A variable component linked to the differential with respect to a threshold (in €/MWh) is then introduced:

$$var(h) = \max\big(0, 180 - PZO_{MWh}(h)\big). \tag{7.12}$$

The model uses parameters (tp_{base}, cap_{var}) determined by plant power brackets, and a fixed addend depending on the macro-area ("zonal factor"). In addition, a reduction linked to grant intensity (e.g. PNRR) is included, modelled through a multiplicative factor. Denoting by F_{zone} the macro-area addend and by f_{grant} the reduction factor, the tariff (in €/MWh) is reduced to the following structure:

$$TIP_{MWh}(h) = \Big(\min\big(cap_{var},\, tp_{base} + var(h)\big) + F_{zone}\Big) \cdot (1 - f_{grant}), \tag{7.13}$$

and therefore:

$$TIP(h) = \frac{TIP_{MWh}(h)}{1000}. \tag{7.14}$$

7.2.6 TIAD component refund profile

Similarly, the simulator describes the TIAD component as an hourly profile $TIAD(h)$ in €/kWh. The following modes are available.

Fixed TIAD.

$$TIAD(h) = \overline{TIAD}. \tag{7.15}$$

TIAD in RSE-like mode (ARERA-like). In this mode the calculation depends on the type of configuration (CER, AUC or absence of CACER). Parameters expressed in €/MWh (e.g. `TRASe`, `BTAU`) are used, which are converted to €/kWh by dividing by 1000.

In the CER case, $TIAD(h)$ is modelled as constant:

$$TIAD(h) = TRASe. \tag{7.16}$$

In the AUC case, $TIAD(h)$ includes a component proportional to PZO weighted by the BT/MT shares:

$$TIAD(h) = TRASe + BTAU + \Big(share_{BT} \cdot Cpr_{BT} + share_{MT} \cdot Cpr_{MT}\Big) \cdot PZO(h), \tag{7.17}$$

where $share_{MT} = 1 - share_{BT}$. If $share_{BT}$ is not specified, the simulator attempts an estimate starting from the members' voltage level; in the absence of information, $share_{BT} = 1$ is assumed.

7.2.7 Tax deduction (Bonus ristrutturazioni): parameterisation

In the simulator, the tax deduction associated with interventions (e.g. FV system, BESS and wind generation, where eligible) is modelled as an additional cash benefit, split into constant annual instalments for a fixed number of years. This choice makes it possible to represent transparently the effect of the deduction on DCF indicators without altering the income statement (PnL)[6] of the project.

[6]PnL (*Profit and Loss statement*): income statement that summarises revenues and costs (and therefore profit/loss) in the period considered.

Denoting by $CAPEX_i^{PV}(0)$, $CAPEX_i^{BESS}(0)$ and $CAPEX_i^{EOL}(0)$ the investments incurred at time zero, the simulator defines the eligible expenditure:

$$CAPEX_i^{\text{elig}} = CAPEX_i^{PV}(0) + CAPEX_i^{BESS}(0) + CAPEX_i^{EOL}(0), \tag{7.18}$$

to which it applies a parametric cap CAP_{det} (expressed in €€ and typically referred to a real-estate unit) and a deduction rate $\tau_{det} \in [0, 1]$:

$$Det_i^{\text{tot}} = \tau_{det} \cdot \min\Big(CAPEX_i^{\text{elig}},\, CAP_{det}\Big)\,. \tag{7.19}$$

The total credit Det_i^{tot} is then spread over N_{det} years (default 10) in constant instalments:

$$D_i(y) = \begin{cases} \dfrac{Det_i^{\text{tot}}}{N_{det}} & \text{se } 1 \leq y \leq N_{det}, \\ 0 & \text{altrimenti.} \end{cases} \tag{7.20}$$

In the current configuration, the simulation assumes sufficient tax capacity to fully use the annual instalment; possible extensions can introduce a capacity constraint (e.g. $D_i(y) = \min(D_i(y), \text{IRPEF_cap}_i(y))$).

7.2.8 Operational remarks and limitations

The modes described above explicitly implement the drivers considered in the simulator (PZO, spread/fee, capacity brackets, macro-area, grant intensity, CACER type and BT/MT split). Any additional factors (e.g. complex contractual conditions, macroeconomic indicators or dynamic tariffs not traceable to PZO) require an extension of the model, to be documented consistently with the source code and the datasets used.

7.3 Purchase costs, sale revenues and incentives

Starting from the hourly energy series, the simulator computes the hourly monetary drivers. The energy purchase cost for member i is:

$$C_i^{\text{buy}}(h) = E_{\text{prel},i}(h)\, p_i^{\text{buy}}(h), \tag{7.21}$$

In addition, the *self-consumption savings* are defined (economic benefit associated with the energy not purchased from the grid):

$$S_i^{\text{aut}}(h) = E_{\text{aut},i}(h)\, p_i^{\text{buy}}(h). \tag{7.22}$$

This quantity is used in the DCF cash model (Section 7.5) as an increase in the flows, while the purchase cost $C_i^{\text{buy}}(h)$ continues to be reported in the PnL.

The revenue associated with injections (if sale is enabled for the member) is:

$$R_i^{\text{sell}}(h) = E_{\text{imm},i}(h)\, p_i^{\text{sell}}(h). \tag{7.23}$$

Incentives are calculated on shared energy at community level. Defining:

$$R_{\text{tot}}^{TIP}(h) = E_{\text{cond}}(h)\, TIP(h), \qquad R_{\text{tot}}^{TIAD}(h) = E_{\text{cond}}(h)\, TIAD(h), \tag{7.24}$$

a parametric allocation rule between consumers and producers is applied, based on coefficient $\alpha_c \in [0, 1]$:

$$R_c^{TIP}(h) = \alpha_c\, R_{\text{tot}}^{TIP}(h), \qquad R_p^{TIP}(h) = (1 - \alpha_c)\, R_{\text{tot}}^{TIP}(h), \tag{7.25}$$

and similarly for $R_c^{TIAD}(h)$ and $R_p^{TIAD}(h)$. In this formulation, α_c represents the policy input defined by the user in the UI (for example 0.45 to allocate 45% to consumers and the remaining 55% to producers).

7.4 Allocation of incentives to members

The allocation of incentives to individual members is performed on an hourly basis and proportional to withdrawal and injection flows. The weights are therefore defined as:

$$w_i^{\text{prel}}(h) = \frac{E_{\text{prel},i}(h)}{\sum_j E_{\text{prel},j}(h)}, \qquad w_i^{\text{imm}}(h) = \frac{E_{\text{imm},i}(h)}{\sum_j E_{\text{imm},j}(h)}, \tag{7.26}$$

with convention $w = 0$ when the denominator is zero. The incentive revenue attributed to member i becomes:

$$R_i^{TIP}(h) = R_c^{TIP}(h)\, w_i^{\text{prel}}(h) + R_p^{TIP}(h)\, w_i^{\text{imm}}(h), \tag{7.27}$$

$$R_i^{TIAD}(h) = R_c^{TIAD}(h)\, w_i^{\text{prel}}(h) + R_p^{TIAD}(h)\, w_i^{\text{imm}}(h). \tag{7.28}$$

This choice ensures consistency with the definition of shared energy: the "consumers" component remunerates members as a function of their contribution to withdrawals, while the "producers" component remunerates members as a function of their contribution to injections. From an implementation standpoint, the quantities $w_i^{\text{prel}}(h)$ and $w_i^{\text{imm}}(h)$ are computed with safe division (*safe division*) to avoid numerical instabilities when, in a given hour, the community has no withdrawals or

no injections.

7.5 Annual income statement and DCF model

The hourly drivers are aggregated on an annual basis to build an income statement and a multi-year cash model. Indicating with year = 1 the year represented by the energy profiles (typically a full year), the totals are defined:

$$C_i^{\text{buy}}(1) = \sum_{h \in \mathcal{H}} C_i^{\text{buy}}(h), \qquad R_i^{\text{sell}}(1) = \sum_{h \in \mathcal{H}} R_i^{\text{sell}}(h), \tag{7.29}$$

$$S_i^{\text{aut}}(1) = \sum_{h \in \mathcal{H}} S_i^{\text{aut}}(h). \tag{7.30}$$

$$R_i^{TIP}(1) = \sum_{h \in \mathcal{H}} R_i^{TIP}(h), \qquad R_i^{TIAD}(1) = \sum_{h \in \mathcal{H}} R_i^{TIAD}(h). \tag{7.31}$$

In the event that the energy run covers a sub-period of the year (for example a quarter), the simulator can normalise the annual totals through a scaling factor based on the number of simulated hours. This option enables preliminary evaluations from partial datasets, while keeping the distinction between observed data and annualisation explicit.

Starting from the annual totals, the simulator builds for each member a simplified income statement, including OPEX[7], any depreciation and a parametric tax component depending on the meter-point class (households/companies) with the possibility of override per member. In addition, the model includes investment costs (CAPEX[8]) at time zero and replacement costs (*replacement*) in specific years, consistently with the declared useful lives of the assets (FV and BESS). In this implementation, a distinction is made between: (i) the income statement (PnL), which includes among operating costs the cost of purchasing energy from the grid; (ii) the cash flow used for the DCF evaluation, built starting from net income but with *incremental* adjustments consistent with the perspective of this work. In particular, the energy purchase cost is kept in the PnL for reporting purposes, but is excluded from the valuation cash flow through an add-back[9]; moreover, (a) the monetary savings linked to self-consumption and (b) any benefit from tax deduction are included.

[7]OPEX (*Operating Expenditure*): recurring operating costs associated with the operation and maintenance of the assets, incurred over the project life.

[8]CAPEX (*Capital Expenditure*): initial investment expenditures for purchasing/building the assets (e.g. FV and BESS), typically incurred at time zero.

[9]In this context, *add-back* refers to re-adding to the valuation cash flow a cost item already accounted for in the PnL, thus neutralising its impact on cash flow (consistently with the adopted incremental perspective).

In compact form, the DCF cash flow is therefore expressed as:

$$\begin{aligned} &CF_i(0) = -CAPEX_i(0), \\ &CF_i(y) = NI_i(y) + DA_i(y) - CAPEX_i^{\text{repl}}(y) + C_i^{\text{buy}}(y) + S_i^{\text{aut}}(y) + D_i(y), \end{aligned} \tag{7.32}$$

where $NI_i(y)$ is the net income of the PnL, $DA_i(y)$ depreciation, $CAPEX_i^{\text{repl}}(y)$ the replacement expenditure in year y, $C_i^{\text{buy}}(y)$ the annual energy purchase cost (add-back in the DCF), $S_i^{\text{aut}}(y)$ the annual self-consumption savings and $D_i(y)$ the annual tax deduction instalment (Section 7.2.7).

Compared to a purely accounting definition of cash flow (without add-back and without additional components), the present formulation tends to increase the flows in the first years and, consequently, the project KPIs (NPV, IRR and payback). Therefore, the results obtained with this setting are not directly comparable with those derived from a DCF that includes the energy purchase cost as a cash outflow.

Starting from the cash flows, the main evaluation indicators (VAN/NPV, TIR/IRR and payback) are defined. In particular, the VAN (or NPV) of member i is given by:

$$NPV_i = \sum_{y=0}^{Y} \frac{CF_i(y)}{(1+r)^y}, \tag{7.33}$$

where r is the discount rate and Y the evaluation horizon. The TIR (or IRR) is defined as the value $r^\star$ that sets the VAN to zero:

$$0 = \sum_{y=0}^{Y} \frac{CF_i(y)}{(1+r^\star)^y}. \tag{7.34}$$

The payback is calculated as the first year in which the cumulative sum of cash flows becomes non-negative. Explicitly, introducing the simple and discounted cumulatives

$$\text{CumCF}_i(y) = \sum_{\tau=0}^{y} CF_i(\tau), \qquad \text{CumDCF}_i(y) = \sum_{\tau=0}^{y} \frac{CF_i(\tau)}{(1+r)^\tau}, \tag{7.35}$$

the simple and discounted payback periods are defined, respectively, as:

$$PB_i = \min\{y \in [0, Y] : \text{CumCF}_i(y) \geq 0\}, \qquad DPB_i = \min\{y \in [0, Y] : \text{CumDCF}_i(y) \geq 0\}. \tag{7.36}$$

7.6 Economic outputs and traceability

Each economic evaluation produces a dedicated output folder (timestamp) in the session directory `economics/outputs/`. The folder contains: (i) result tables (`pnl_by_member.csv`, `cashflow_by_member.csv`, `kpis_by_member.csv`, and the corresponding aggregated versions); (ii) a `manifest.json` file that lists the available artefacts and facilitates programmatic discovery of the results; (iii) the complete snapshot of the economic assumptions, saved in the same folder alongside the results.

In particular, the `cashflow_by_member.csv` table explicitly includes the additional components introduced in the DCF perspective, including self-consumption savings (`savings_autocons`) and the annual tax deduction instalment (`tax_detraction`), in support of traceability and comparison across scenarios.

Similarly to balancing, an economy-level session register (`economics/runs_index.jsonl`) and a pointer to the active economic run (`economics/active_run.txt`) are also maintained. The run record includes references to the main files and fingerprints of the inputs (energy scenario, selected energy run and economic assumption pack). This mechanism makes the dependency chain between energy and economics explicit and enables reproducible comparisons across different tariff, tax or investment assumptions.

7.7 Optimisation of BESS sizing as a function of NPV

Within the simulator, BESS optimisation is not implemented as an independent *sweep* for each member, but rather as an incremental allocation at community level based on a *greedy algorithm*. The objective is to maximise the CER NPV (community VAN) by distributing storage capacity among a subset of candidate members.

7.7.1 Allocation problem and assumptions

Consider a set of enabled members $\mathcal{M}$ (present in the economic evaluation) and a subset of candidates $\mathcal{C} \subseteq \mathcal{M}$ on which storage sizing is allowed. The allocation is described by a vector of aggregate capacities $\mathbf{C} = \{C_i\}_{i \in \mathcal{C}}$, with $C_i \geq 0$ (kWh). The energy–economic model defines an objective function:

$$\mathrm{NPV}_{\mathrm{CER}}(\mathbf{C}) \,, \tag{7.37}$$

obtained by running (i) energy balancing with BESS for the involved members and recomputing the community quantities (in particular E_{cond}), and (ii) the economic evaluation with tariff, tax and investment assumptions frozen, varying only the BESS size.

In practice, a quantisation of capacity is introduced through a fixed step ΔC (kWh) and, optionally, a constraint on the total allocable $C_{\max}$.

7.7.2 Greedy capacity-increment algorithm

The algorithm starts from a base configuration $\mathbf{C}^{(0)} = \mathbf{0}$ and iterates by adding an increment ΔC to the single member that maximises the marginal NPV increase given the current state. At generic iteration t:

1. For each candidate $i \in \mathcal{C}$ a trial $\mathbf{C}^{(t)} + \Delta C\, \mathbf{e}_i$ is built.
2. For the trial, the hourly flows of member i are recomputed with the BESS model (Chapter 6, Section 6.6); these series replace those of the current energy run and the aggregated CER quantities are recomputed.
3. The economic evaluation is rerun keeping all assumptions unchanged, except for the initial BESS capacity of member i (CAPEX/OPEX consistent with the new size).
4. The marginal increase is estimated:

$$\Delta\text{NPV}^{(t)}_{\text{CER},i} = \text{NPV}_{\text{CER}}\big(\mathbf{C}^{(t)} + \Delta C\, \mathbf{e}_i\big) - \text{NPV}_{\text{CER}}\big(\mathbf{C}^{(t)}\big)\,. \tag{7.38}$$

Then,

$$i^\star = \arg\max_{i \in \mathcal{C}} \Delta\text{NPV}^{(t)}_{\text{CER},i}\,, \tag{7.39}$$

and the allocation $\mathbf{C}^{(t+1)} = \mathbf{C}^{(t)} + \Delta C\, \mathbf{e}_{i^\star}$ is accepted. In this way, the optimisation accounts for interactions and saturations: each trial is evaluated against the "current" run that already incorporates the increments accepted in previous iterations.

7.7.3 Stopping criteria and outputs

The algorithm stops when at least one of the following conditions occurs: (i) exceeding the total limit $C_{\max}$; (ii) non-positive marginal increase $\Delta\text{NPV}^{(t)}_{\text{CER},i^\star} \leq 0$ (if stop on negative marginality is enabled); (iii) absence of admissible candidates.

Outputs are saved in a folder `opt_{timestamp}` and include: (a) an iterative trace of the allocation (step, selected member, ΔNPV and sizes before/after); (b) a

per-member summary with final capacity $C_i^\star$ and cumulative contribution to NPV; (c) a stepwise curve compatible with dashboard visualisation; (d) metadata with run parameters (ΔC, $C_{\max}$, candidates, starting energy run). The algorithm is implemented in module `cer_core/optimization/bess_greedy.py` and is orchestrated by the Streamlit economic evaluation page.

7.7.4 Considerations

The greedy procedure is a heuristic: it does not guarantee the global optimum of $\mathrm{NPV}_{\mathrm{CER}}(\mathbf{C})$, but provides a computationally manageable and interpretable solution ("packet" allocation), particularly suitable for scenarios with many members and with a non-separable objective function due to shared energy E_{cond}.

Chapter 8

Case study: dataset construction and scenario set-up

This chapter applies the simulator developed in the previous chapters to a real case study located in the Municipality of Doberdò del Lago (Province of Gorizia, North-East Italy). The objective is twofold:

- to build an *end-to-end coherent* dataset (consumption and generation) starting from information collected in the field and from public sources, realigning it to the format required by the simulator (temporal resolution, base year, data structures and temporal conventions);
- to *qualitatively* validate the consumption profiles synthesised by the bottom-up model (Chapter 5) through a comparison with provincial average profiles published by ARERA/Portale Consumi (average hourly withdrawals by month and day type).

The base year chosen for the energy processing of the case study is 2023, both for consumption and for the estimated photovoltaic generation, so as to maintain temporal consistency with the session conventions (Chapter 3) and to make downstream results directly comparable (balancing and economic indicators). The ARERA dataset used for the shape benchmark (provincial average profile) instead refers to the most recent year available at the time of analysis (2024); this information is used exclusively as a reference *shape* and is then realigned to the base-year calendar and resolutions.

8.1 Primary data collection: in-person interviews and plant survey

The first phase of the work concerned the collection of primary data through in-person interviews at four residential meter points (hereafter *User 1–4*, anonymised). The on-site survey made it possible to acquire two sets of information:

- **Electricity consumption:** annual consumption (kWh/anno) and information on usage habits (occupancy hours, use of appliances, any energy-intensive uses), necessary to parameterise the bottom-up demand model (Chapter 5).
- **Plant equipment (where present):** installed FV power (kWp) and actual plant configuration (orientation/tilt and installation constraints observed during the site inspection). The presence of electrochemical storage (BESS) and the associated nominal capacity (kWh) was also verified, where available, for the subsequent balancing phase (Chapter 6).

In the analysed sample there are two *prosumer* meter points (equipped with FV generation) and two *consumer* meter points (consumption only). The main collected characteristics are summarised in Table 8.1.

Table 8.1: Main characteristics of the interviewed meter points (base year 2023).

User	Role	Annual consumption [kWh]	Installed FV [kWp]	BESS [kWh]	Power class (ARERA)
User 1	Consumer	2000	–	–	3–4,5 kW
User 2	Prosumer	6000	15,5	12	> 6 kW
User 3	Prosumer	4500	8,25	4	> 6 kW
User 4	Consumer	2700	–	–	3–4,5 kW

8.2 Estimation of photovoltaic generation: PVGIS and actual roof configuration

For the prosumer users (User 2 and User 3) complete and homogeneous historical series of measured generation were not available. Consistently with the approach adopted in the generation module (Chapter 4), FV generation was therefore estimated via PVGIS (JRC – European Commission), obtaining hourly time series of FV output and associated meteorological variables.

For each plant the following were set:

- **location:** Doberdò del Lago;
- **year:** 2023 (base year of the case study);
- **nominal power:** equal to the surveyed installed power (Table 8.1);
- **geometric parameters:** orientation and tilt consistent with the actual installation observed on the roof;
- **system assumptions:** losses and operational assumptions according to the model conventions (Chapter 4).

No wind plants are present in the case study; therefore the entire contribution of local renewable generation is attributed to photovoltaic. The PVGIS hourly series were subsequently realigned to the session temporal conventions (Chapter 3) and, where necessary, brought to the T_{15} resolution of the balancing (Chapter 6), preserving energy consistency per interval.

8.3 ARERA consumption curves as benchmark: motivation and source

The demand profile in the simulator is generated through a bottom-up model (Chapter 5), hence parametric and dependent on the information collected in the field. To reduce the risk of introducing profiles that are unrealistic in terms of shape (intra-daily and seasonal distribution), an external validation was introduced: comparing the profiles synthesised by the simulator with curves derived from a *third* institutional source.

For this purpose, data published by ARERA/Portale Consumi relating to the average electricity consumption of residential meter points[31] were used, filterable by:

- Region/Province,
- year,
- power class,
- market type,
- residence.

Among the available quantities, the portal provides profiles of **average hourly withdrawal** (in kWh) for residential customers treated on an hourly basis, aggregated by province. Consistently with the setting of the case study, data relating to the Province of Gorizia and to the power classes corresponding to the interviewed meter points (3–4,5 kW and >6 kW) were downloaded. As anticipated at the outset, the ARERA dataset used for the benchmark refers to 2024: in this application it is used as a shape reference and subsequently realigned to the base-year calendar.

8.4 From the ARERA *month* × *day type* data to an annual series compatible with the simulator

ARERA/Portale Consumi files typically provide an average hourly profile differentiated by:

- month,
- day type (*weekday*, *Saturday*, *Sunday*),
- hour (`Ora1` ... `Ora24`).

To directly compare these profiles with those produced by the simulator (continuous series on a regular time grid), a reconstruction procedure was implemented that transforms the ARERA information into an annual hourly series and, if necessary, converts it to the T_{15} resolution.

8.4.1 Cleaning and restructuring of the files (filter + pivot)

The first phase consists of:

1. conversion of downloaded files (possibly in `xlsx` format) into tabular `csv` format;
2. filtering for **Province** = **Gorizia** and for the demographic fields consistent with the sample (e.g. residence, where available);
3. normalisation of column names and restructuring into *tidy* form, obtaining a table indexed by (*mese*, *tipo_giorno*) with 24 hourly columns (`Ora1` ... `Ora24`) containing the provincial average hourly withdrawal (kWh).

8.4.2 Construction of the annual calendar and mapping of the *day type*

An hourly calendar is built for the entire base year (2023), obtaining a grid of 8760 timestamps in a convention consistent with the session (Chapter 3). For each timestamp h (hour) the following are computed:

- month $m(h) \in \{1, \ldots, 12\}$,
- hour of day $k(h) \in \{0, \ldots, 23\}$,
- day of week and corresponding classification into:
  - Mon–Fri $\rightarrow$ *weekday*,
  - Sat $\rightarrow$ *Saturday*,
  - Sun $\rightarrow$ *Sunday*.

This classification makes it possible to associate to each hour of the year the correct ARERA row ($mese \times tipo_giorno$) and the corresponding hourly column.

8.4.3 Join and reconstruction of the hourly series

The calendar is joined to the ARERA profiles suitably brought into *long* form $(m, tipo_giorno, k) \mapsto$ kWh, obtaining a continuous hourly series:

$$E^{T_H}_{\text{ARERA}}(h), \qquad \forall h \in \mathcal{H}_{2023}, \tag{8.1}$$

where $\mathcal{H}_{2023}$ denotes the set of hours of the base year and $E^{T_H}_{\text{ARERA}}(h)$ is the provincial average energy withdrawn in hour h (kWh).

Note that this series represents a *provincial-average* reconstruction (not a point measurement of the individual meter point); for this reason it is used as a *shape* benchmark and not as the quantitative ground-truth for the individual customer.

8.4.4 Scaling to the meter point annual consumption: from a profile "shape" to a profile "user"

To make the ARERA benchmark comparable with each interviewed meter point, the hourly series is scaled by imposing that annual energy matches the observed annual

consumption E_{utente}:

$$E^{T_H}_{\text{ARERA,scal}}(h) = E^{T_H}_{\text{ARERA}}(h) \frac{E_{\text{utente}}}{\sum\limits_{h \in \mathcal{H}_{2023}} E^{T_H}_{\text{ARERA}}(h)}. \tag{8.2}$$

In this way, the comparison between simulated profile and ARERA benchmark primarily concerns the *temporal distribution* of energy (intra-daily and seasonal), while keeping the total annual energy of the individual user consistent.

8.4.5 Conversion $T_H \to T_{15}$ for direct comparison

The simulator produces 15-minute demand profiles (T_{15}). To enable a direct comparison on the same time grid, the scaled hourly series is converted to 15 minutes through a discretisation *piecewise-constant* that preserves energy:

$$E^{T_{15}}_{\text{ARERA,scal}}(t) = \frac{1}{4} E^{T_H}_{\text{ARERA,scal}}(h), \qquad \forall t \in h, \tag{8.3}$$

where t denotes a 15-minute interval and $t \in h$ means that interval t belongs to hour h.

This choice makes:

- the ARERA profile and the simulated one comparable in T_{15};
- it possible to perform comparisons also after aggregation to hourly (sum of quarter-hours) or on specific time windows (e.g. month, day type, time bands).

8.5 Expected output of the data phase and integration into the scenario

At the end of the phases described, the case study has the following artefacts, temporally aligned and ready for executing energy balancing and calculating shared energy (Chapter 6):

- **consumption series for each user** generated by the simulator bottom-up model (resolution T_{15}, base year 2023), parameterised on the data collected in the interview;
- **FV generation series** for prosumer users (PVGIS, base year 2023), realigned to the session conventions and made compatible with the simulator data structures;

- **"benchmark" consumption series** derived from ARERA/Portale Consumi (Province of Gorizia, consistent power class), reconstructed as an annual series, scaled to the user's annual consumption and converted to the T_{15} resolution for qualitative comparison.

This phase therefore constitutes the end point of dataset construction: all required series are available in tabular form, consistent in base year and resolution, and can be persisted within the scenario perimeter according to the audit and reproducibility conventions introduced in Chapters 2–3.

8.6 Energy balancing results: comparison between ARERA profiles and simulated profiles

Once the consumption and generation series have been built and temporally aligned (Sections 8.1–8.5), the next step consists in executing the energy balancing described in Chapter 6, with calculation of quantities at individual member and community level: self-consumption, withdrawals and injections, as well as hourly shared energy E_{cond}.

To isolate the effect due to the *shape* of the demand profile, two runs identical in community configuration and renewable generation (FV from PVGIS) were compared, differing only in the source of consumption:

- **Run ARERA (benchmark)**: demand built starting from ARERA provincial average profiles (Province of Gorizia, consistent power class), reconstructed as an annual series, scaled to the user's annual energy and converted to T_{15} (Section 8.4).

- **Simulated run (bottom-up)**: demand generated by the simulator bottom-up model (Chapter 5), parameterised on the information collected in the interview and scaled to the observed annual energy.

In both cases, the same surveyed plant equipment was kept (Table 8.1), including BESS storage where present (12 kWh for User 2 and 4 kWh for User 3), so as to compare results with the same assets and vary only the temporal distribution of loads.

8.6.1 Community KPIs

Table 8.2 summarises the main energy KPIs at CER level obtained from the two runs. In both cases, the overall annual energy produced and consumed remains substantially

unchanged (by construction of the dataset and scaling), whereas quantities depending on the temporal alignment between demand and generation vary.

Table 8.2: Comparison of community KPIs (ARERA run vs simulated run).

KPI (CER)	**ARERA**	**Simulated**	**Δ (Sim–ARERA)**
$E_{\text{prod,tot}}$ [kWh]	26329,9	26329,9	0,0
$E_{\text{load,tot}}$ [kWh]	15200,0	15200,7	+0,7
$E_{\text{cond,tot}}$ [kWh]	1136,3	1469,7	+333,4
$E_{\text{import,tot}}$ [kWh]	6663,9	5636,9	-1026,9
$E_{\text{export,tot}}$ [kWh]	17400,8	16422,6	-978,2
`share_on_production`	0,0432	0,0558	+0,0126
`coverage_on_load`	0,0748	0,0967	+0,0219

The run with simulated consumption yields a higher annual shared energy (about +333 kWh, equal to $\sim$ +29% with respect to the benchmark), and consequently shows:

- an increase in the share of production that is shared within the community (`share_on_production`: 0,0432 $\rightarrow$ 0,0558);
- an increase in load coverage through shared energy (`coverage_on_load`: 0,0748 $\rightarrow$ 0,0967);
- a reduction in both total grid import (-1027 kWh) and total export (-978 kWh), consistent with greater local use of FV generation.

8.6.2 Member KPIs and impact on prosumers

The effect of the different demand profile is concentrated on the prosumer members, in particular on User 2, who has the largest installed FV power. Table 8.3 reports the main quantities for the two prosumers (consumer users do not have local generation and therefore have null self-consumption and self-sufficiency equal to zero).

The most evident result concerns the **self-sufficiency** of User 2, which increases from 0,799 (ARERA benchmark) to 0,862 (simulated profile). This increase is consistent with the rise in annual self-consumption (about +380 kWh) and with the simultaneous reduction in grid withdrawal (~ -380 kWh), given the same annual FV generation. A similar behaviour, although slightly smaller in absolute terms, is also observed for User 3.

Table 8.3: Comparison of KPIs for prosumer members (ARERA run vs simulated run).

Member	KPI	ARERA	Simulated
User 2	$E_{\text{aut,tot}}$ [kWh]	4794,6	5174,7
	$E_{\text{imm,tot}}$ [kWh]	11449,4	11128,6
	$E_{\text{prel,tot}}$ [kWh]	1205,4	825,7
	`self_consumption`	0,290	0,313
	`self_sufficiency`	0,799	0,862
User 3	$E_{\text{aut,tot}}$ [kWh]	2605,2	2919,4
	$E_{\text{imm,tot}}$ [kWh]	7087,7	6763,7
	$E_{\text{prel,tot}}$ [kWh]	1894,8	1580,8
	`self_consumption`	0,266	0,298
	`self_sufficiency`	0,579	0,649

8.6.3 Discussion: interpretation of differences and "daytime" bias of the simulated profile

Overall, the two runs produce *consistent* results and quantitative differences remain limited relative to the annual energies involved. This suggests that the dataset built from ARERA provides a coherent benchmark, while the bottom-up model generates plausible profiles from the standpoint of overall energy.

The systematic difference observed (higher shared energy and higher self-sufficiency in the simulated run) can be attributed to the **greater temporal correlation** between load and FV generation introduced by the demand model. In particular, the interview-based parameterisation highlighted the tendency of consumers to concentrate the use of the most demanding loads (e.g. washing machine, dishwasher, heat pump) during **daylight hours**, also with the implicit objective of maximising the use of locally produced energy. This effect emerges especially for User 2: being the member with the larger FV plant size, shifting load toward daylight hours produces a more marked increase in self-consumption and a reduction in grid withdrawal, consequently improving self-sufficiency (0,79 → 0,86).

From a methodological standpoint, this result is useful because:

- it confirms that the *shape* of the demand profile (not only annual energy) is determinant for E_{cond} and for self-consumption/self-sufficiency indicators;
- it highlights a possible *optimistic bias* of the simulated profile when declared habits tend to overestimate the ability to shift loads (behavioural flexibility).

Looking ahead, calibrating the bottom-up model on real measurements (smart meter) would allow quantification of this bias and refinement of the estimate of the

flexibility actually available, improving the reliability of shared energy estimates and of downstream economic KPIs.

8.7 Economic-financial evaluation: price assumptions, incentives and assets

The economic valuation of the energy flows obtained from the balancing (Section 8.6) is performed through the evaluation module described in Chapter 7. In this phase, the hourly energy profiles (withdrawals, injections and shared energy) are transformed into annual economic quantities (costs, revenues and incentives) and subsequently projected over a DCF horizon.

8.7.1 Purchase and sale prices

For the **energy purchase** component, a time-band tariff (**F1, F2, F3**) differentiated by meter point was adopted, with the objective of using unit prices that are *realistic* and consistent with residential consumption profiles. Prices were obtained through the Portale Offerte (Italy's offer portal)[32]: for each user, consumption data were entered and, starting from the economic values estimated for a set of configurations/offers, an **equivalent unit price** in €/kWh (per band) was calculated as the average of the obtained values. In this way, the valuation of residual grid withdrawal (`cost_buy`) reflects the order of magnitude and the time-band structure typical of residential bills.

For the **energy sale** component, the **PZO of the NORD macro-area** was assumed as the sale price. The PZO profile was acquired through the GME API Service[33]. Due to licence and usage constraints associated with the API Service, the raw downloaded data are not redistributed in this document[34].

In the DCF, the purchase and sale **unit prices** are projected over 20 years by applying an annual inflation rate of **2%**.

8.7.2 Incentives: TIP and TIAD

Incentives are calculated on shared energy on an hourly basis $E_{\text{cond}}(h)$, consistently with the definition and outputs of the balancing module (Chapter 6). In the case study:

- the **TIP** (`rev_tip`) is calculated according to the setting implemented in the simulator (`rse_decree`);

- the **TIAD** (`rev_tiad`) is calculated with a parameterisation consistent with a CER in the NORD macro-area (implementation `rse_arera`).

As regards the **incentivised capacity**, both FV plants of the prosumers were considered fully eligible (no reduction of incentivisable capacity). Moreover, no capital grants were considered (no non-repayable contribution or plant account). In the absence of specific indications on future regulatory scenarios, incentives are kept *constant* over the analysis horizon.

8.7.3 Assets and CAPEX/OPEX assumptions

In this version of the case study, assets are valued by including an initial investment (**CAPEX**) and annual operating costs (**OPEX**). The adopted unit values, discussed with RSE, are:

- **Photovoltaic (PV):** CAPEX = 1100 €/kW, OPEX = 18 €/kW/anno;
- **Battery (BESS):** CAPEX = 600 €/kWh, OPEX = 0,01 €/kWh/anno.

For each prosumer i, the initial CAPEX is calculated as:

$$\mathrm{CAPEX}_i = 1100 \cdot P_{\mathrm{PV},i} \; + \; 600 \cdot C_{\mathrm{BESS},i},$$

where $P_{\mathrm{PV},i}$ is the installed FV power (kW) and $C_{\mathrm{BESS},i}$ the nominal storage capacity (kWh), consistent with the data in Table 8.1. Similarly, annual OPEX is:

$$\mathrm{OPEX}_i = 18 \cdot P_{\mathrm{PV},i} \; + \; 0{,}01 \cdot C_{\mathrm{BESS},i}.$$

8.7.4 Finance and taxation

The DCF horizon is 20 years, with discount rate $r = 3\%$. In the time projection the following inflation assumptions were applied:

- **energy purchase prices:** +2% per year;
- **energy sale prices:** +2% per year;
- **OPEX:** +2% per year.

It also considers a **50% deduction rate** on eligible CAPEX (assumption consistent with the incentive framework currently available at the time of analysis). Consistently with the parameterisation described in Section 7.2.7, the deduction is not treated as a reduction of the initial net CAPEX; instead, it is modelled as an

additional cash benefit split into annual instalments $D_i(y)$ in the first N_{det} years of the horizon. In other words, CAPEX at time zero remains unchanged and the deduction enters the cash flows as an increase of the cash flow in subsequent years. In the set of simulations reported here, the contribution is set to zero in the export files (`tax_detraction`=0), therefore the reported KPIs are conservative with respect to a scenario with the deduction active. Ordinary taxation is otherwise disabled (no income/profit tax modelled in the DCF).

8.8 Economic-financial results: comparison between simulated profiles and ARERA benchmark

This section compares the economic-financial outputs obtained with (i) *simulated* consumption profiles (bottom-up model, Chapter 5) and (ii) scaled *ARERA* profiles (Section 8.4), keeping **assets, price assumptions and incentive mechanisms unchanged** (Section 8.7). The reported values are calculated starting from the simulator export files (cash flow: `Cash flow totale_dati_simulati.csv` and `Cash flow totale_dati_ARERA.csv`; income statement: `PnL totale_dati_simulati.csv` and `PnL totale_dati_ARERA.csv`).

8.8.1 Overall KPIs

Table 8.4 summarises the main DCF KPIs at community level. The KPIs should therefore be understood as *conservative* relative to a possible scenario in which the tax deduction (Section 7.2.7) is actually accounted for as a cash benefit.

Table 8.4: Comparison of overall economic-financial KPIs (ARERA run vs simulated run).

KPI (totale CER)	**ARERA**	**Simulated**	**Δ (Sim−ARERA)**
NPV / VAN [€]	35 875,861	39 334,589	+3 458,728
IRR / TIR [–]	0,080	0,086	+0,006
Payback [years]	11	10	-1
Discounted Payback [years]	11	10	-1

The run with simulated data shows a systematic improvement in KPIs: the VAN increase is +3 458,728 €(about +9,6% compared to the benchmark), while the TIR increases from 8,0% to 8,6%. Consistently, the payback period decreases by about one year (Payback and Discounted Payback). The result is consistent with what was discussed in Section 8.6: given the same FV generation, the simulated profiles

increase shared energy and self-consumption (Table 8.2 and Table 8.3), improving overall economic valuation.

8.8.2 Income statement (PnL): main drivers

The variation in economic KPIs can be read through the main items of the annual income statement: sale revenues (rev_sell), incentive revenues (rev_tip and rev_tiad) and grid purchase cost (cost_buy). Table 8.5 provides a comparison for the base year (Year 1).

Table 8.5: Income statement comparison (Year 1): ARERA run vs simulated run.

Item (Year 1)	**ARERA [€]**	**Simulated [€]**	**Δ [€]**
rev_sell	1 762,685	1 698,023	-64,663
rev_tip	146,421	189,814	+43,392
rev_tiad	13,522	17,489	+3,967
cost_buy	2 020,742	1 856,598	-164,144
EBITDA	-621,614	-474,773	+146,841

It can be observed that, in the simulated run, incentive revenues (rev_tip and rev_tiad) increase, consistently with higher shared energy, while both the grid purchase cost (cost_buy) and sale revenues (rev_sell) decrease. The latter effect can be interpreted as a direct consequence of lower exported energy: a larger share of production is in fact valued internally through self-consumption/sharing.

On a cumulative basis over the analysis horizon (sum of years 1–20), the dynamics is confirmed: the higher incentive revenues and the reduction in cost_buy more than compensate for the reduction in rev_sell, improving overall EBITDA. For completeness, Table 8.6 reports the sums of the main items (years 1–20).

Table 8.6: Sum of the main income statement items (years 1–20): ARERA run vs simulated run.

Item (years 1–20)	**ARERA [€]**	**Simulated [€]**	**Δ [€]**
rev_sell	42 828,619	41 257,483	-1 571,136
rev_tip	2 928,421	3 796,270	+867,849
rev_tiad	270,441	349,789	+79,348
cost_buy	49 098,716	45 110,451	-3 988,265
opex	12 719,673	12 719,673	0,000
EBITDA	-15 790,908	-12 426,581	+3 364,326

8.8.3 Cash flows and decomposition of the ΔVAN

The community annual cash flow (years 1–20) is built, consistently with Chapter 7, as the sum of: self-consumption savings (`savings_autocons`), sale revenues (rev_sell), incentive revenues (rev_tip and rev_tiad), OPEX and (any) replacements (`capex_replacement`). In the exported dataset, the only modelled replacement is 2,850 €at Year 12, identical in both runs.

The VAN difference between simulated and ARERA (+3 458,728 €) can be fully explained by decomposing the discounted contributions of the cash-flow components. Table 8.7 summarises this decomposition.

Table 8.7: Decomposition of the (discounted) ΔVAN between simulated run and ARERA run.

Component	**Contribution to ΔVAN [€]**
Higher self-consumption savings (`savings_autocons`)	+4 046,196
Higher incentive revenues (TIP+TIAD)	+899,241
Lower sale revenues (rev_sell)	-1 486,709
Total	+3 458,728

The main driver is therefore the increase in self-consumption savings, followed by the increase in incentive revenues, while the reduction in sale revenues partially offsets the overall benefit. This also explains the earlier break-even point: in the ARERA run the discounted cumulative remains negative at Year 10 (-1 178,557 €) and becomes positive at Year 11 (2 548,692 €), whereas in the simulated run it is already positive at Year 10 (467,393 €).

8.8.4 Comparison with existing simulators and chapter closing

The Payback of 10 years obtained in the run with simulated data is in line with the orders of magnitude typically returned by simulation tools available for energy-sharing configurations, such as the *CACER_Simulator* tool developed by RSE[30]. Overall, the case study confirms that the **realistic representation of hourly demand profiles** has a direct impact on economic feasibility, as it modifies shared energy and self-consumption and therefore (i) incentive revenues and (ii) savings from avoided purchase of energy from the grid.

Chapter 9

Conclusions and future developments

This work presented the design and implementation of a modular end-to-end simulator for the analysis of Renewable Energy Communities (CER), capable of reconstructing (i) the energy flows of individual members and of the community and (ii) the corresponding economic and financial results, ensuring reproducibility and traceability through filesystem persistence and run versioning. Following the case study and the comparisons carried out, this section summarises the main extension margins of the simulator and concludes with a synthesis of the contribution provided.

9.1 Future developments of the simulator

The most interesting future developments, in continuity with the architecture and with the modules already implemented, can be organised into three main lines: (i) extension of the generation component and of its geometric representation, (ii) enrichment of the demand model with new loads and new behaviours, (iii) introduction of broader optimisation procedures that include not only storage but also generation assets.

9.1.1 Generation: data acquisition and automatic 3D render generation via convolutional networks

In the current simulator, the definition of photovoltaic areas and the generation of 3D renders of the roof and panels support the qualitative validation of the configuration (packing, geometric constraints, spacing), but require explicit input in the interface and a manual construction of geometries. A natural evolution is to automate (partially or totally) 3D reconstruction and/or the extraction of installable surfaces through models based on convolutional neural networks (CNNs) and, more generally,

on computer vision.

The application objective can be formulated as: given a set of observations (for instance roof images, ground photographs, or satellite views), automatically estimate a 3D representation and/or a set of "installable" polygons to be used as input for the PV module. In practical terms, the pipeline could include:

- **surface segmentation and classification**: identification of roof pitches, obstacles (chimneys, dormers), permanent shaded areas, and non-installable margins;
- **geometry and orientation estimation**: tilt and azimuth of roof pitches, possibly with associated uncertainty;
- **3D reconstruction**: transition from images to a mesh (or simplified model) consistent with the renderer already envisaged;
- **derivation of canonical inputs**: conversion of the result into GeoJSON/polygons and geometric parameters compatible with the packing logic and with calls to PVGIS.

The main criticality of this direction is not so much software integration (which would naturally graft onto the "Producers" branch), as the **availability of data for training and validation**. In a real context, obtaining supervised datasets (images → installable labels and/or 3D mesh "ground truth") requires substantial work: data collection, annotation, definition of labelling standards, and management of variability (materials, weather conditions, image quality, occlusions). A pragmatic approach can envisage an initial phase with:

- **synthetic data**: generation of roofs and simulated scenarios to create automatically labelled examples;
- **transfer learning**: reuse of pre-trained models for segmentation/monodepth, adapted with a reduced but targeted dataset;
- **human-in-the-loop**: a workflow in which the model proposes areas/meshes and the user corrects in the UI, reducing annotation cost and maintaining auditability.

The added value, on the simulator side, would be twofold: (i) reduce the burden of manual geometry entry and (ii) increase the consistency between the "real asset" and the simulated configuration, making both the installable capacity and the visual validation more reliable.

9.1.2 Consumers: integration of new loads (e.g. electric vehicles) with a bottom-up approach

The consumption model was built with a **bottom-up** logic: the total profile derives from the sum of simple and interpretable components (baseload, occupancy, cooking, laundry, air conditioning), each with explicit parameters and saved intermediate outputs. This choice makes it natural to extend the simulator by introducing **new systems/appliances** without overturning the architecture.

In particular, EV integration can be implemented consistently with the modules already present by adopting a minimal pipeline:

- definition (from real data or distributions) of **arrival/departure times** and **daily mileage**;
- conversion km → kWh via vehicle efficiency and definition of the **charging requirement**;
- generation of the power profile respecting **EVSE power** and **availability constraints**;
- optional **shifting of the charging cycle** towards hours of higher PV generation or towards lower-cost time bands (F1–F3), keeping the decision criterion transparent.

This "block-based" setting makes it possible to increase demand realism while maintaining **control and auditability**: each new load is traceable and can be enabled/disabled for sensitivity analyses and what-if scenarios.

9.1.3 Optimisation: extension from the battery alone to the optimisation of PV and wind systems

In the current version, optimisation is focused on the sizing/allocation of the BESS as a function of NPV, through a greedy heuristic that recalculates balancing and economics for candidate configurations. A natural evolution is to extend optimisation by including **generation assets** (photovoltaic and wind), with objectives and constraints compatible with CER feasibility studies.

Examples of optimisable problems include:

- **PV sizing**: choice of installed capacity (or number of modules) and, where applicable, selection of the areas actually used;

- **orientation/tilt**: search for optimal (or discretised) tilt/azimuth to maximise self-consumption, shared energy, or NPV;
- **wind sizing**: choice of turbine model and size, with constraints on hub height, losses, and site availability;
- **integrated optimisation**: co-design of PV, wind, and BESS, taking into account interactions among demand profiles, generation, and valorisation mechanisms (shared energy, sales, savings from self-consumption).

From a methodological standpoint, this extension requires:

- an explicit objective function (NPV, payback period, shared energy, load coverage, or multi-objective formulations);
- realistic constraints (surfaces, permitting limits, budget, technical constraints);
- robust computational strategies (grid search, metaheuristics, or Bayesian optimisation) to manage the search space without exploding the number of runs.

A strength of the simulator is that the "scenario → energy run → economic run" structure, with stable data contracts and versioned outputs, makes these iterations intrinsically traceable and comparable: each candidate solution is an inspectable run, not an "opaque" configuration.

9.2 Conclusions: usefulness of the simulator and modularity for reuse and specialised analyses

Overall, the developed simulator is useful as a **tool for understanding and feasibility assessment** of energy communities, because it allows one to:

- quantify and explain key energy flows (withdrawals, injections, self-consumption, shared energy) on a time basis consistent with regulation;
- translate such flows into economic and financial results comparable across scenarios, with explicit assumptions saved alongside the runs;
- highlight the potential benefits of the CER (savings from self-consumption, incentives on shared energy, revenues from sales) and their sensitivity with respect to inputs.

These characteristics are particularly relevant in the design phase, when the objective is not only to estimate a final value, but to understand **why** a certain asset or a certain behaviour (for example load shifting) modifies shared energy and, therefore, economic convenience.

A further concluding element concerns the **modularity** of the simulator. The separation between orchestration (UI) and domain logic (core), together with artefact persistence and data contracts between modules, allows the system to be decomposed and sub-parts to be reused in more specific contexts. Concrete examples of reuse include:

- isolating the consumption subsystem to develop targeted analyses on **tariff choices** and "bill vs profile" simulations (for example comparing F1–F3 structures or indexed offers);
- using the producers branch to estimate, in a reproducible manner, **PV/wind producibility** on different locations and configurations, even independently of the CER part;
- replacing inputs (external profiles) while keeping balancing and economics unchanged, thanks to the integration bridge and the definition of canonical scenario inputs.

In summary, the work shows that a simulator designed with attention to reproducibility, traceability, and modularity is not only a "KPI calculator", but an operational support to explore scenarios, motivate design decisions, and communicate clearly the benefits and trade-offs associated with an energy community.

Bibliography

[1] European Parliament and Council, *Directive (eu) 2018/2001 of the european parliament and of the council of 11 december 2018 on the promotion of the use of energy from renewable sources (red ii)*, Official Journal of the European Union, 2018.

[2] European Parliament and Council, *Directive (eu) 2019/944 of the european parliament and of the council of 5 june 2019 on common rules for the internal market for electricity*, Official Journal of the European Union, 2019.

[3] "Consolidated text on distributed self-consumption (tiad)," ARERA – Autorità di Regolazione per Energia Reti e Ambiente, 2024, Annex A to Resolution 727/2022/R/eel, as supplemented and amended by Resolution 15/2024/R/eel. [Online]. Available: `https://www.arera.it/fileadmin/allegati/docs/22/727-22TIAD.pdf`

[4] "Cacer decree and tiad – operating rules for access to the service for distributed self-consumption and to the pnrr grant (annex 1)," GSE – Gestore dei Servizi Energetici, 2025, Version of 16 July 2025. [Online]. Available: `https://www.gse.it/documenti_site/Documenti%20GSE/Servizi%20per%20te/AUTOCONSUMO/Gruppi%20di%20autoconsumatori%20e%20comunita%20di%20energia%20rinnovabile/Regole%20e%20procedure/ALLEGATO%201%20Regole%20Operative%20CACER.pdf`

[5] Ministry of the Environment and Energy Security, *Decree of the ministry of the environment and energy security, 7 december 2023, n. 414 (cacer decree)*, 2023. [Online]. Available: `https://www.tb.camcom.gov.it/uploads/AMB/normative/Decreto%20MASE%20n.414%20del%2007.12.2023.pdf`

[6] Open-Meteo. "Open-meteo: Free weather api. "[Online]. Available: `https://open-meteo.com/`

[7] Joint Research Centre, European Commission. "Pvgis – photovoltaic geographical information system. "[Online]. Available: `https://joint-research-`

centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis_en

[8] European Commission, Joint Research Centre. "Pvgis api: Non-interactive service. "[Online]. Available: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/getting-started-pvgis/api-non-interactive-service_en

[9] European Commission, Joint Research Centre. "Pvgis tool: Hourly radiation / hourly pv output. "[Online]. Available: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/pvgis-tools/hourly-radiation_en

[10] J. F. Manwell, J. G. McGowan, and A. L. Rogers, *Wind Energy Explained: Theory, Design and Application*, 2nd ed. John Wiley & Sons, 2009, ISBN: 9780470015001. [Online]. Available: https://www.wiley.com/en-gb/Wind%2BEnergy%2BExplained%3A%2BTheory%2C%2BDesign%2Band%2BApplication%2C%2B2nd%2BEdition-p-9780470015001

[11] J. Freeman, P. Gilman, J. Jorgenson, and T. Ferguson, "Reference manual for the system advisor model's wind power performance model," National Renewable Energy Laboratory (NREL), Tech. Rep. NREL/TP-6A20-60570, 2014. DOI: 10.2172/1150800 [Online]. Available: https://docs.nrel.gov/docs/fy14osti/60570.pdf

[12] C. G. Justus and A. Mikhail, "Height variation of wind speed and wind distributions statistics," *Geophysical Research Letters*, vol. 3, no. 5, pp. 261–264, 1976. DOI: 10.1029/GL003I005P00261

[13] International Electrotechnical Commission (IEC), *Iec 61400-12-1:2022 wind energy generation systems – part 12-1: Power performance measurements of electricity producing wind turbines*, IEC Webstore, 2022. [Online]. Available: https://webstore.iec.ch/en/publication/68499

[14] A. Capasso, W. Grattieri, R. Lamedica, and A. Prudenzi, "A bottom-up approach to residential load modeling," *IEEE Transactions on Power Systems*, vol. 9, no. 2, pp. 957–964, 1994. DOI: 10.1109/59.317650

[15] I. Richardson, M. Thomson, and D. Infield, "Domestic electricity use: A high-resolution energy demand model," *Energy and Buildings*, vol. 42, no. 10, pp. 1878–1887, 2010. DOI: 10.1016/j.enbuild.2010.05.023

[16] N. Pflugradt, P. Stenzel, L. Kotzur, and D. Stolten, "Loadprofilegenerator: An agent-based behavior simulation for generating residential load profiles," *Journal of Open Source Software*, vol. 7, no. 71, p. 3574, 2022. DOI: 10.21105/joss.03574

[17] G. Besagni, L. Premoli Vilà, and M. Borgarello, "Italian household load profiles: A monitoring campaign," *Buildings*, vol. 10, no. 12, p. 217, 2020. DOI: 10.3390/buildings10120217

[18] G. Azzolini et al., "Update of methodologies for estimating energy consumption in the residential sector and analysis of the data from the tax deductions database using the developed methodology," ENEA – Italian National Agency for New Technologies, Energy and Sustainable Economic Development, Tech. Rep. RdS/PAR 2017/086, 2017, Ministry of Economic Development, System Research.

[19] E. McKenna and M. Thomson, "High-resolution stochastic integrated thermal–electrical domestic demand model," *Applied Energy*, vol. 165, pp. 445–461, 2016. DOI: 10.1016/j.apenergy.2015.12.032

[20] J. Widén and E. Wäckelgård, "A high-resolution stochastic model of domestic activity patterns and electricity demand," *Applied Energy*, vol. 87, no. 6, pp. 1880–1892, 2010. DOI: 10.1016/j.apenergy.2009.11.006

[21] I. Richardson, M. Thomson, D. Infield, and C. Clifford, "A high-resolution domestic building occupancy model for energy demand simulations," *Energy and Buildings*, vol. 40, no. 8, pp. 1560–1566, 2008. DOI: 10.1016/j.enbuild.2008.02.006

[22] M. Stokes, M. Rylatt, and K. Lomas, "A simple model of domestic lighting demand," *Energy and Buildings*, vol. 36, no. 2, pp. 103–116, 2004. DOI: 10.1016/j.enbuild.2003.10.007

[23] I. Richardson, M. Thomson, D. Infield, and A. Delahunty, "Domestic lighting: A high-resolution energy demand model," *Energy and Buildings*, vol. 41, no. 7, pp. 781–789, 2009. DOI: 10.1016/j.enbuild.2009.02.010

[24] European Commission, *Commission regulation (eu) 2019/2022 of 1 october 2019 laying down ecodesign requirements for household dishwashers pursuant to directive 2009/125/ec*, Official Journal of the European Union, 2019. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2019/2022/oj/eng

[25] European Commission, *Commission delegated regulation (eu) 2019/2017 of 11 march 2019 with regard to energy labelling of household dishwashers and repealing delegated regulation (eu) no 1059/2010*, Official Journal of the European Union, 2019. [Online]. Available: https://eur-lex.europa.eu/eli/reg_del/2019/2017/oj/eng

[26] European Commission, *Commission regulation (eu) 2019/2023 of 1 october 2019 laying down ecodesign requirements for household washing machines and household washer-dryers pursuant to directive 2009/125/ec*, Official Journal of the European Union, 2019. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2019/2023/oj/eng

[27] European Commission, *Commission delegated regulation (eu) 2019/2014 of 11 march 2019 with regard to energy labelling of household washing machines and household washer-dryers and repealing delegated regulation (eu) no 1061/2010 and directive 96/60/ec*, Official Journal of the European Union, 2019. [Online]. Available: https://eur-lex.europa.eu/eli/reg_del/2019/2014/oj/eng

[28] J. Vivian, A. Zarrella, G. Emmi, and M. De Carli, "Analysis of simplified lumped-capacitance models to simulate the thermal behaviour of buildings," in *Pre-prints of BSA 2017 – Building Simulation Applications*, 2017.

[29] H. Pieper, I. Krupenski, W. B. Markussen, T. S. Ommen, A. Siirde, and A. Volkova, "Method of linear approximation of cop for heat pumps and chillers based on thermodynamic modelling and off-design operation," *Energy*, vol. 230, p. 120 743, 2021. DOI: 10.1016/j.energy.2021.120743

[30] RSE-CoLabs, *Cacer_simulator: Simulation tool for renewable energy sharing configurations*, 2025. [Online]. Available: https://github.com/RSE-CoLabs/CACER_Simulator

[31] ARERA – Autorità di Regolazione per Energia Reti e Ambiente. "Analysis of domestic customers' consumption. "[Online]. Available: https://www.arera.it/dati-e-statistiche/dettaglio/analisi-dei-consumi-dei-clienti-domestici

[32] Acquirente Unico. "The portale offerte: Comparison of electricity and gas tariffs and prices. "[Online]. Available: https://www.ilportaleofferte.it/portaleOfferte/it/confronto-tariffe-prezzi-luce-gas.page

[33] Gestore dei Mercati Energetici (GME). "Api interface. "[Online]. Available: https://www.mercatoelettrico.org/it-it/Home/InterfacciaAPI

[34] "General conditions of use of market data of proprietà of the gestore dei mercati energetici s.p.a. accessible via api service," Gestore dei Mercati Energetici (GME), 2025. [Online]. Available: https://www.mercatoelettrico.org/Portals/0/Documents/it-IT/20251015CONDIZIONI_GENERALI_API.pdf